



วิชาการพัฒนาแอปพลิเคชันบนอุปกรณ์เคลื่อนที่

Application Development for Mobile Devices



อ. พิชัย จอดพิมาย
ห้อง 970

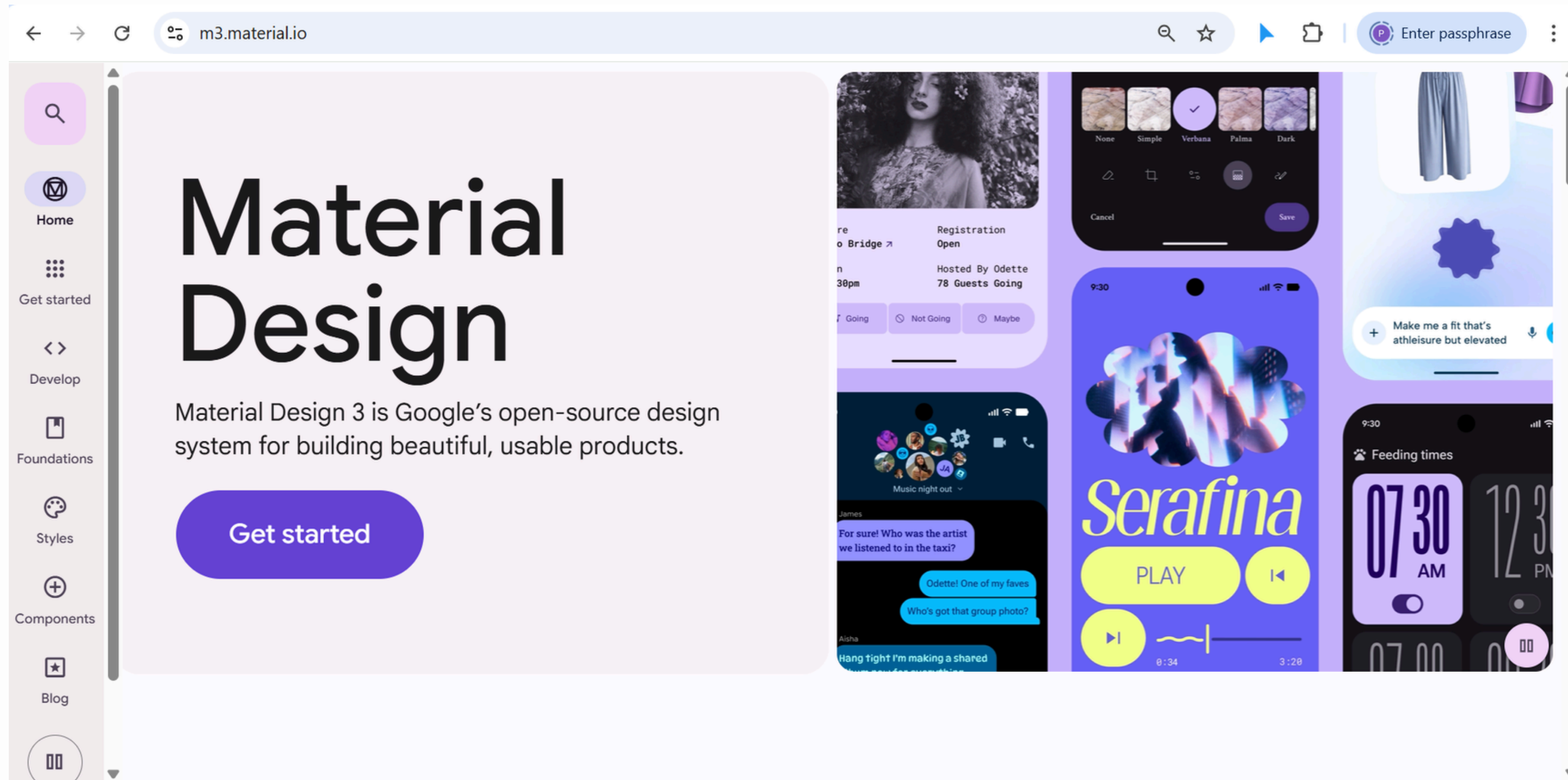
pichai_jo@rmutto.ac.th



การพัฒนาฟรอนต์เอนด์

1. การออกแบบตามแนวทางของ Material Design
2. การใช้งาน Flutter Widgets
3. การเปลี่ยนชื่อและไอคอนของ Launcher
4. Flutter Libraries
5. ตัวอย่างการออกแบบระบบซื้อขายออนไลน์
6. หน้าหลัก
7. หน้าเข้าสู่ระบบ
8. หน้าโปรไฟล์

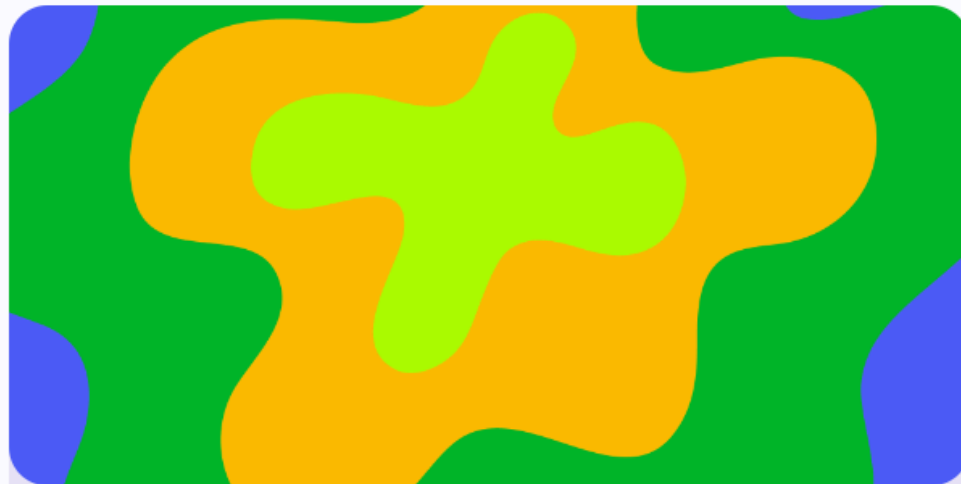
1. การออกแบบตามแนวทางของ Material Design



<https://m3.material.io/>

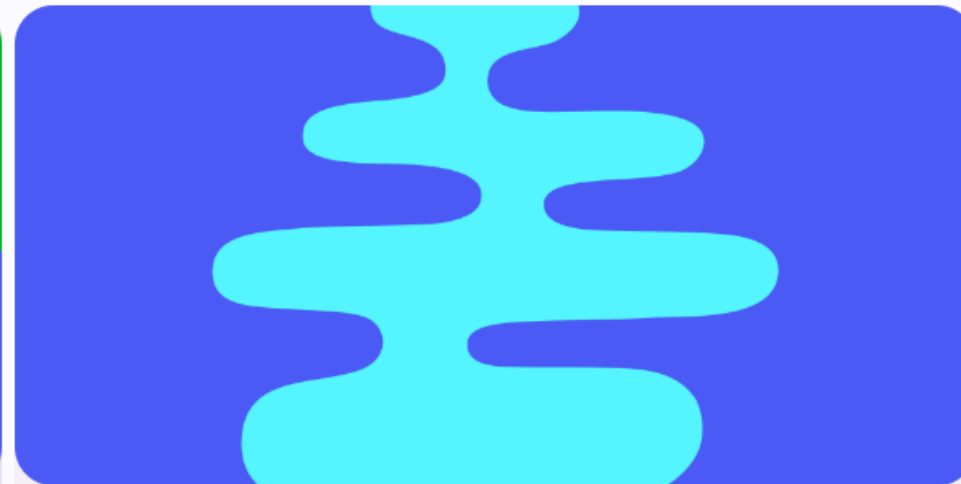
Styles

Styles are the visual aspects of a UI that give it a distinct look and feel. They can be customized by changing your Material theme.



Color

Color is used to express style and communicate meaning. With dynamic color, Material puts personal color preferences and individual needs at the forefront of systematic color application.



Elevation

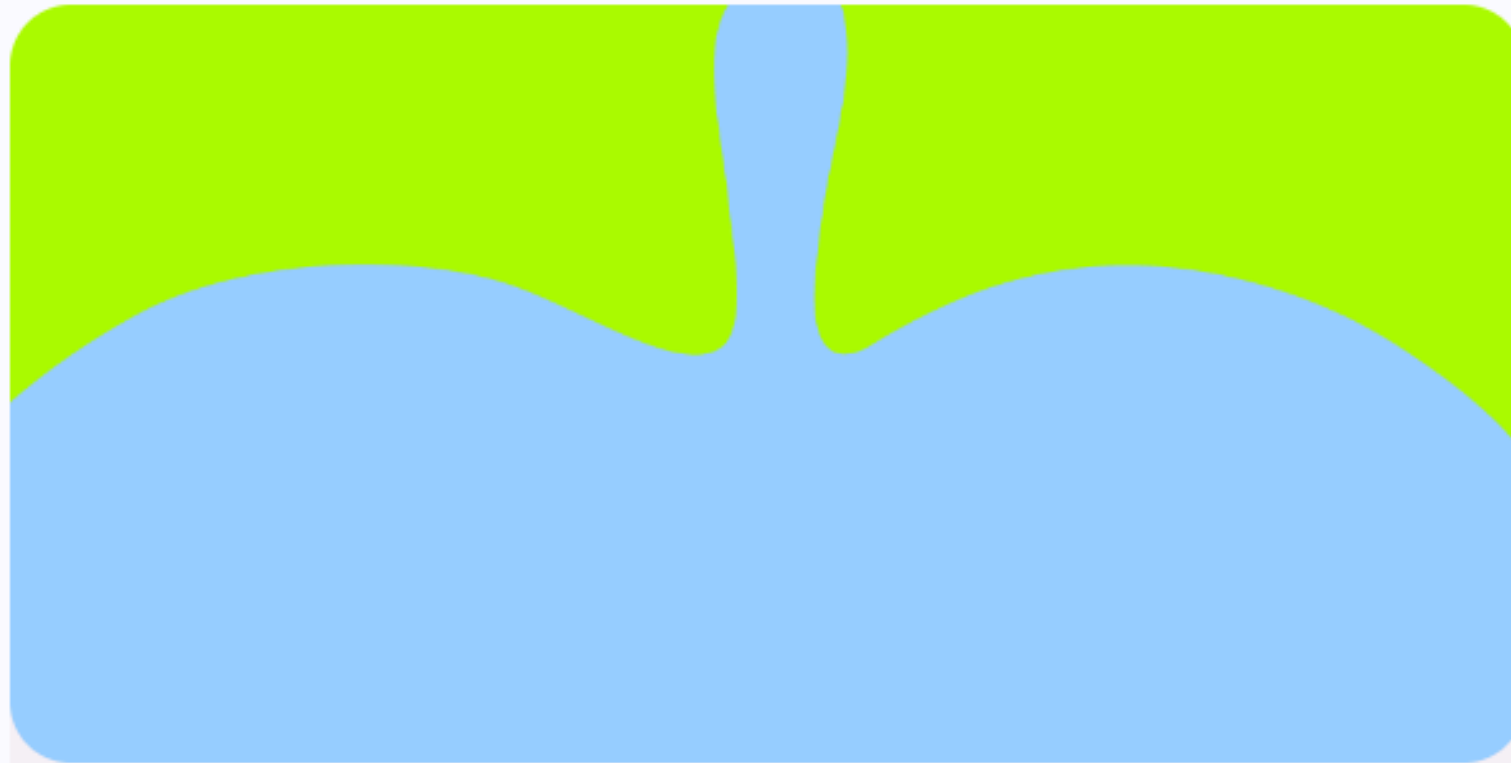
Elevation is the relative distance between two surfaces along the z-axis. All surfaces and components have elevation values.



Icons

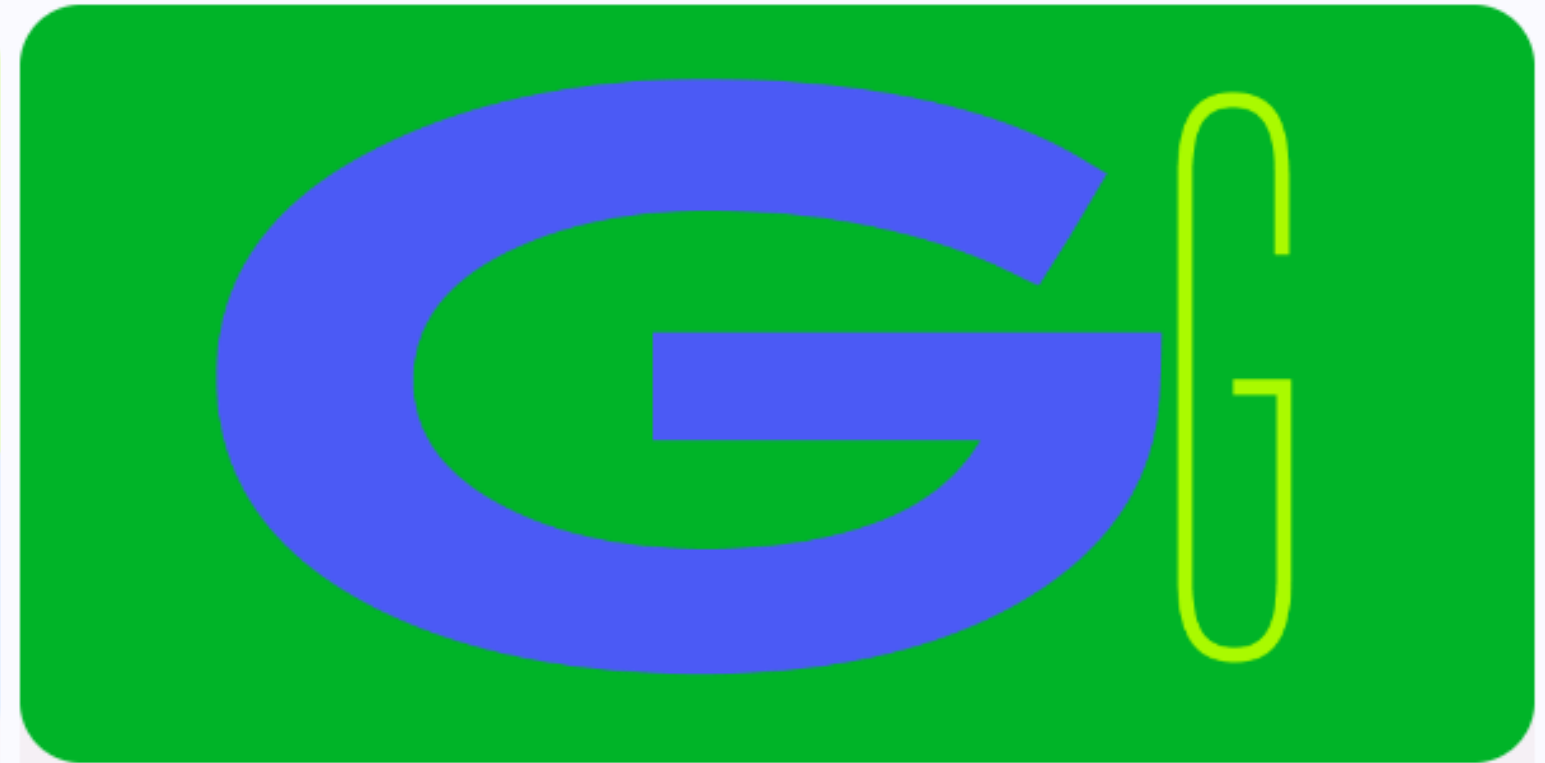


Motion



Shape

The shape scale defines the style of container corners, offering a range of roundedness from square to fully circular.



Typography

Use typography to make writing legible and beautiful. Material's default type scale includes contrasting and flexible styles to support a wide range of use cases.

Primary

ชื่อ

ใช้เมื่อไร

primary

ปุ่มหลัก, AppBar, FAB

onPrimary

ข้อความ/ไอคอนบน primary

primaryContainer

section / card ที่เกี่ยวกับ action หลัก

onPrimaryContainer

ข้อความบน primaryContainer

Secondary

ชื่อ

ใช้เมื่อไร

secondary

ปุ่มรอง, chip

onSecondary

ข้อความบน secondary

secondaryContainer

พื้นหลัง soft สำหรับ content รอง

onSecondaryContainer

ข้อความบน secondaryContainer

Tertiary

ชื่อ

ใช้เมื่อไร

tertiary

visualization, badge, tag

onTertiary

ข้อความบน tertiary

tertiaryContainer

background ของข้อมูลเสริม

onTertiaryContainer

ข้อความบน tertiaryContainer

Error

ชื่อ

ใช้เมื่อไร

error

error state, validation

onError

ข้อความบน error

errorContainer

error แบบอ่อน

onErrorContainer

ข้อความบน errorContainer



Surface (หัวใจของ Material 3)

ชื่อ

ใช้เมื่อไร

surface

พื้นหลังหลักของแอป (Scaffold)

onSurface

ข้อความ/ไอคอนหลัก

surfaceVariant

list, card, divider area

onSurfaceVariant

ข้อความรอง

Surface Containers (Layering)

ชื่อ	ใช้เมื่อไร
surfaceContainerLowest	พื้นหลังชั้นล่างสุด
surfaceContainerLow	Card ธรรมดา
surfaceContainer	Container ทั่วไป
surfaceContainerHigh	Dialog / Elevated section
surfaceContainerHighest	Modal / BottomSheet
surfaceDim	พื้นหลัง dim (inactive)
surfaceBright	พื้นหลังสว่างพิเศษ



Inverse

ชื่อ

ใช้เมื่อไร

`inverseSurface`

Snackbar / Toast

`onInverseSurface`

ข้อความบน `inverseSurface`

`inversePrimary`

FAB หรือ highlight บน dark surface



Outline / Utility

ชื่อ

ใช้เมื่อไร

outline

border ปกติ

outlineVariant

divider อ่อน

shadow

เงา (ใช้ลดลงใน M3)

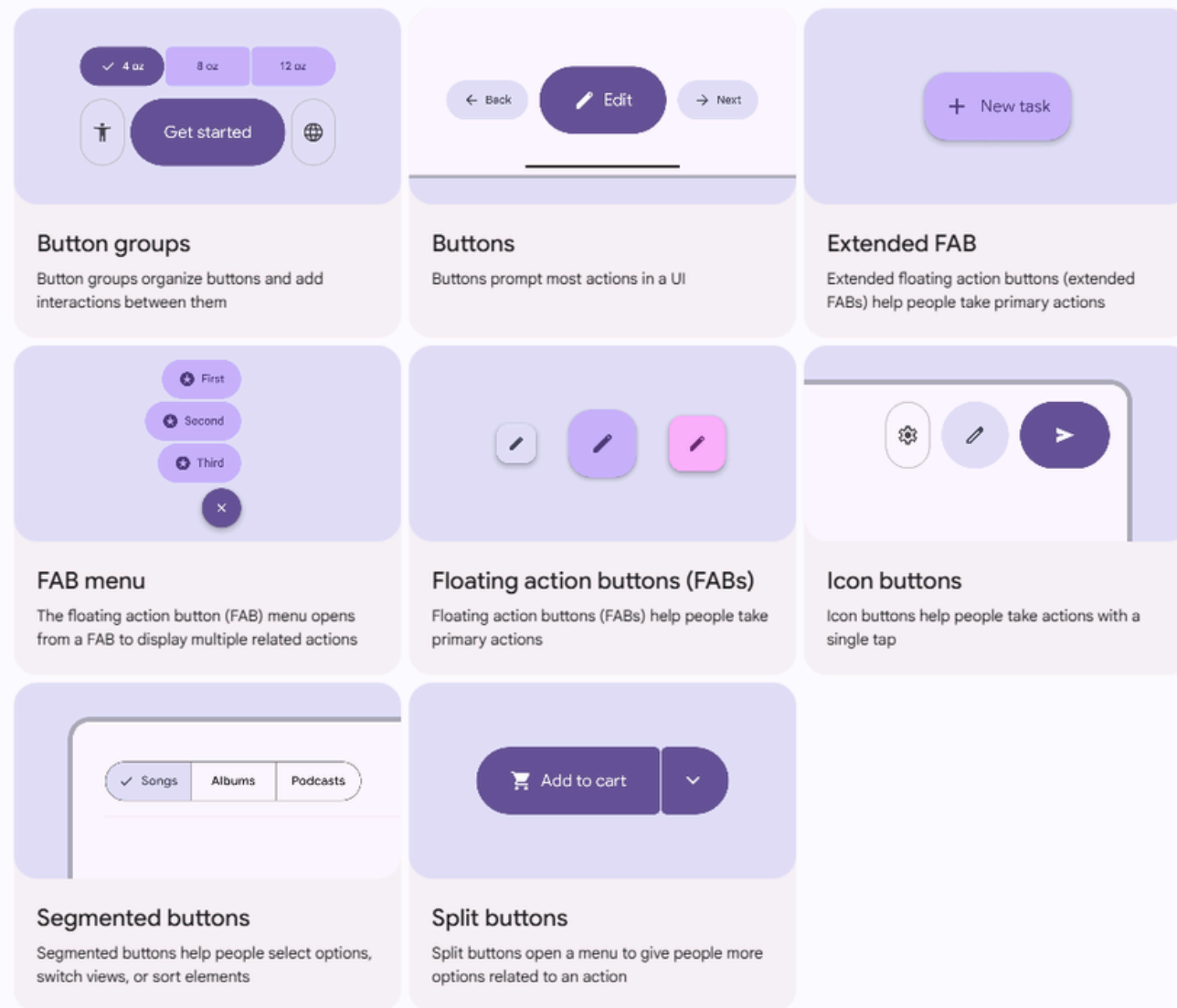
scrim

พื้นหลังมืดเมื่อเปิด modal

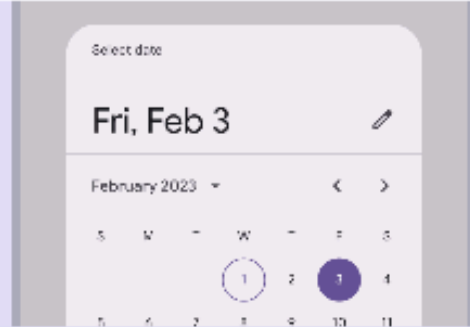
Components

Components are interactive building blocks for creating a user interface. They can be organized into categories based on their purpose: Action, containment, communication, navigation, selection, and text input.

Buttons

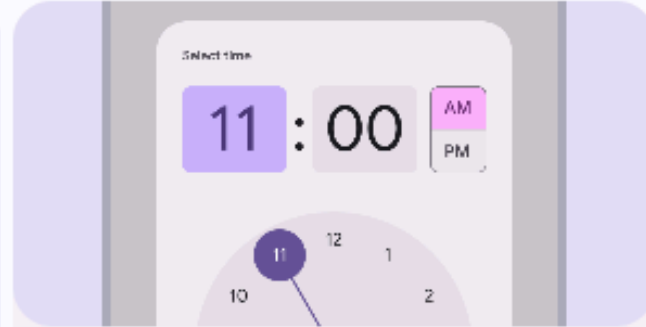


Date & time pickers



Date pickers

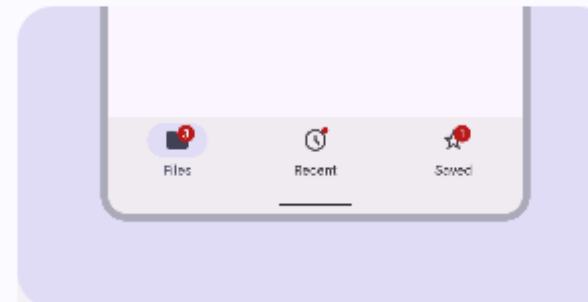
Date pickers let people select a date, or a range of dates



Time pickers

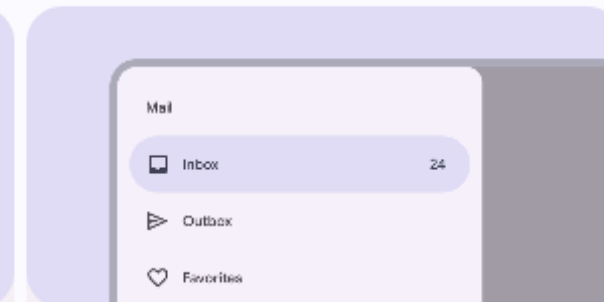
Time pickers help people select and set a specific time

Navigation



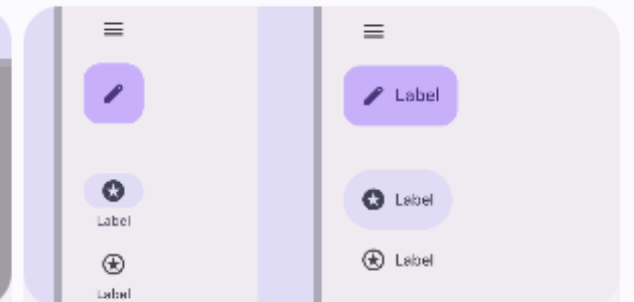
Navigation bar

Navigation bars let people switch between UI views on smaller devices



Navigation drawer

Navigation drawers let people switch between UI views on larger devices



Navigation rail

Navigation rails let people switch between UI views on mid-sized devices

Loading & progress



Loading indicator

Loading indicators show the progress of a process for a short wait time



Progress Indicators

Progress indicators show the status of a process in real time

Sheets



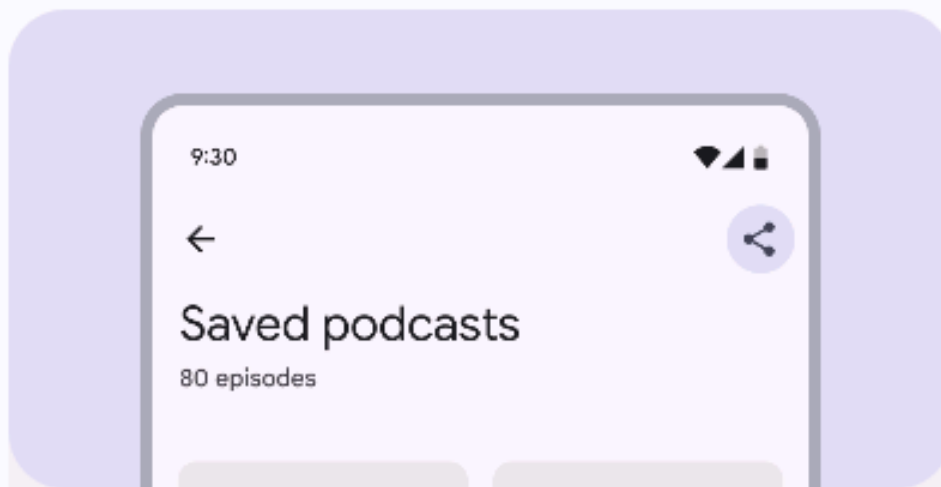
Bottom sheets

Bottom sheets show secondary content anchored to the bottom of the screen



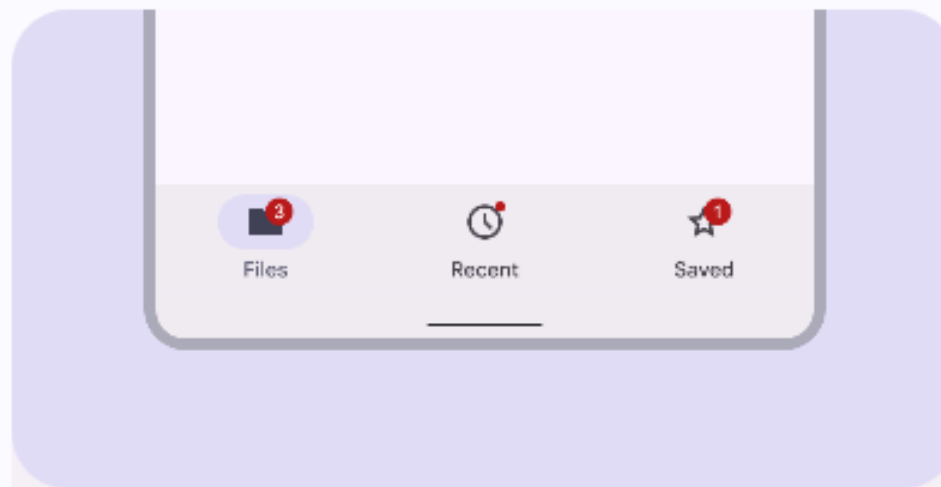
Side sheets

Side sheets show secondary content anchored to the side of the screen



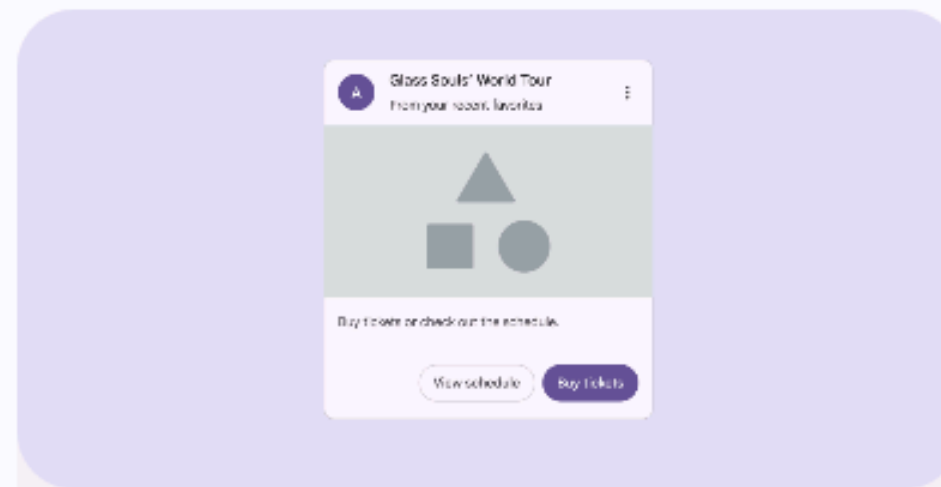
App bars

App bars are placed at the top of the screen to help people navigate through a product



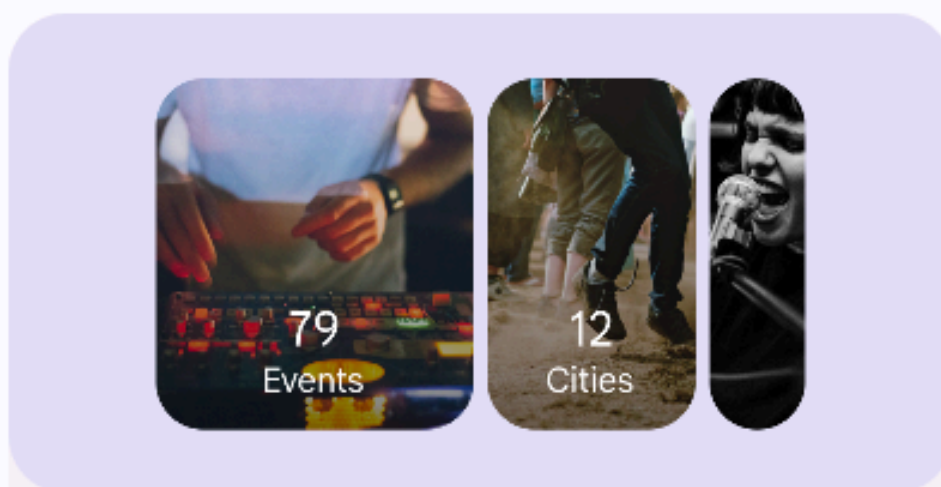
Badges

Badges show notifications, counts, or status information on navigation items and icons



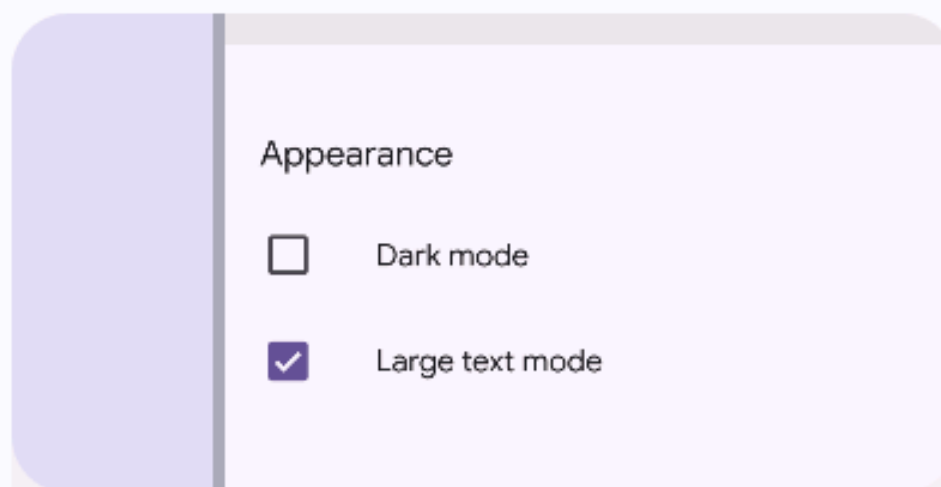
Cards

Cards display content and actions about a single subject



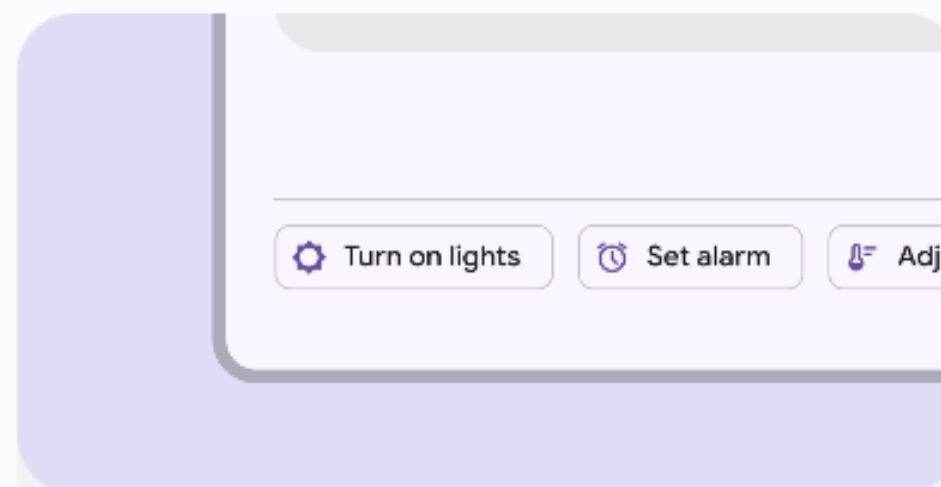
Carousel

Carousels show a collection of items that can be scrolled on and off the screen



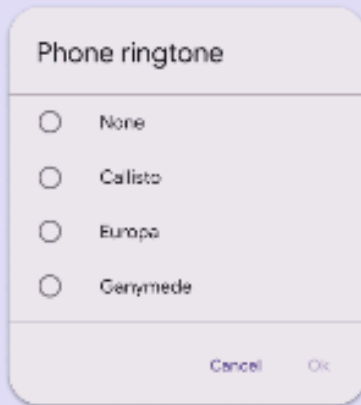
Checkbox

Checkboxes let users select one or more items from a list, or turn an item on or off



Chips

Chips help people enter information, make selections, filter content, or trigger actions



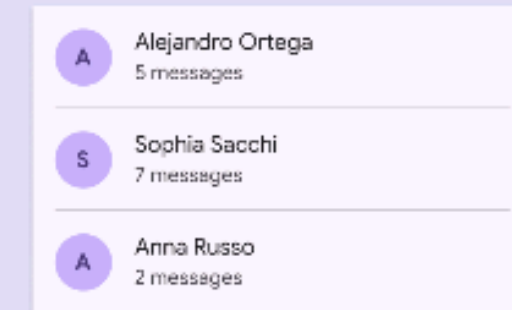
Dialogs

Dialogs provide important prompts in a user flow



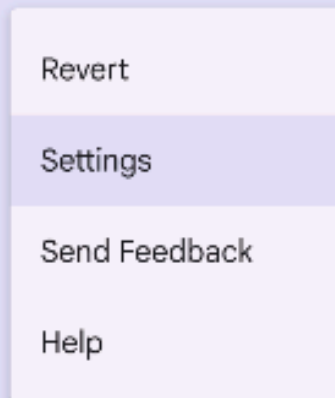
Divider

Dividers are thin lines that group content in lists or other containers



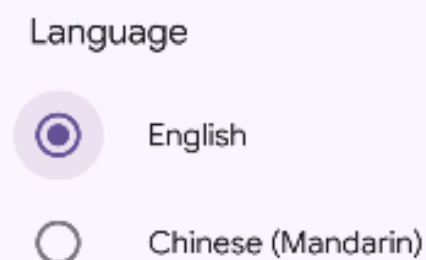
Lists

Lists are continuous, vertical indexes of text and images



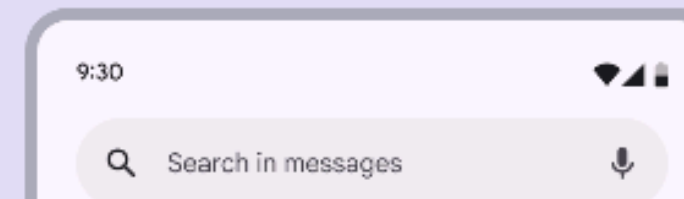
Menus

Menus display a list of choices on a temporary surface



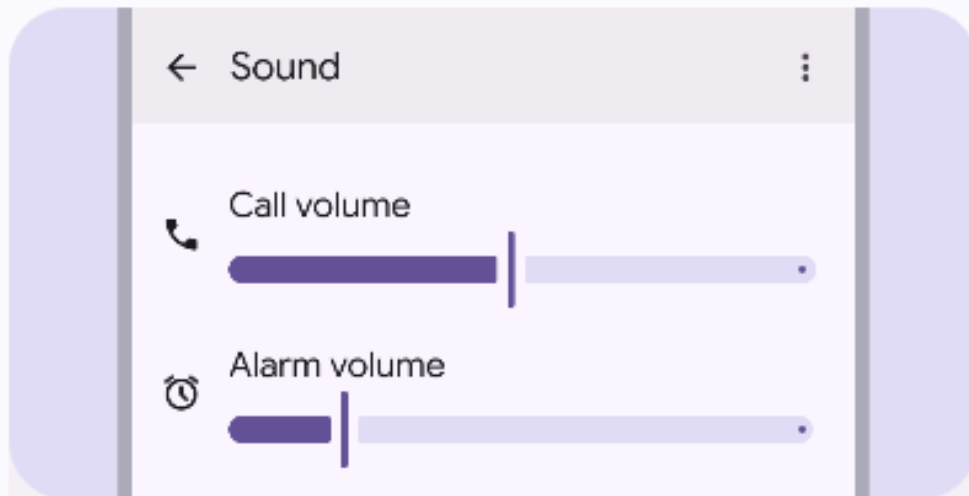
Radio button

Radio buttons let people select one option from a set of options



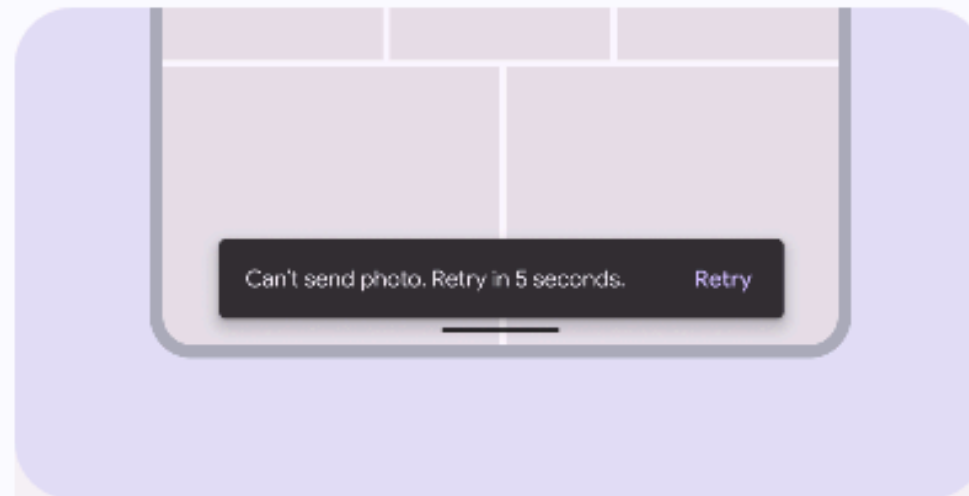
Search

Search lets people enter a keyword or phrase to get relevant information



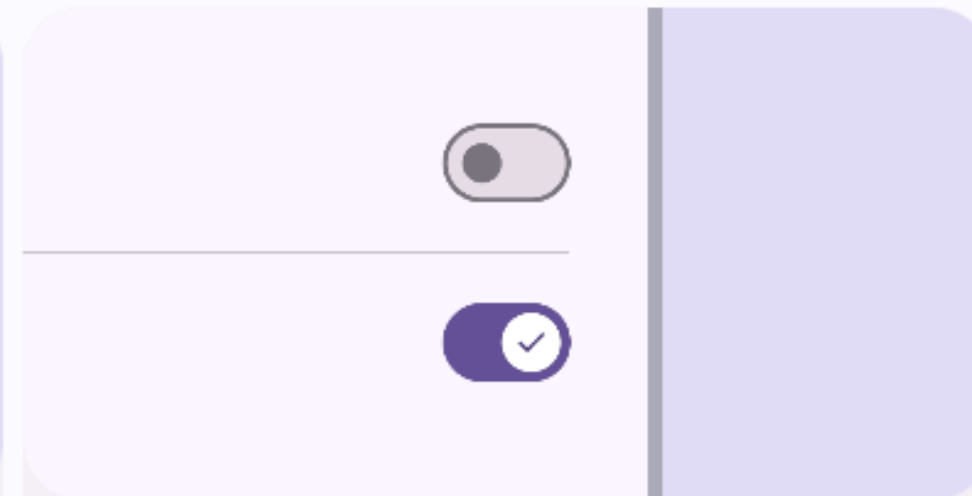
Sliders

Sliders allow users to make selections from a range of values



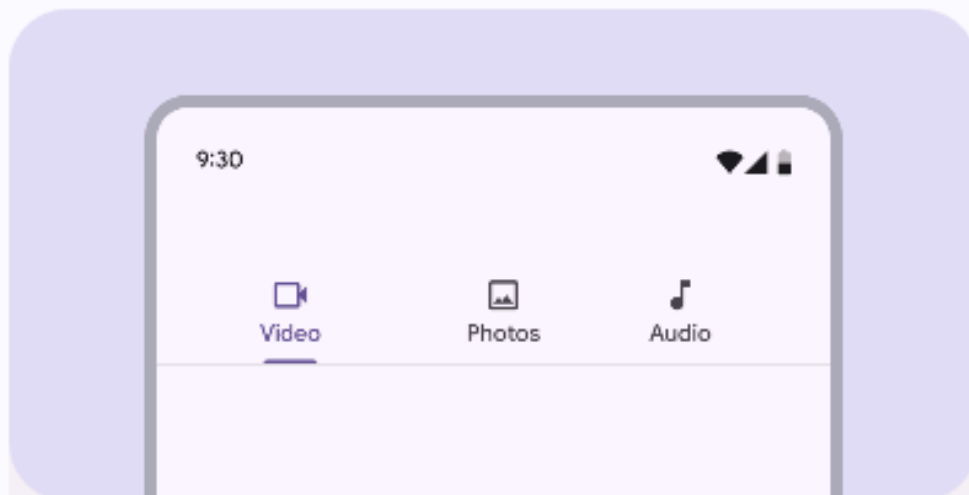
Snackbar

Snackbars show short updates about app processes at the bottom of the screen



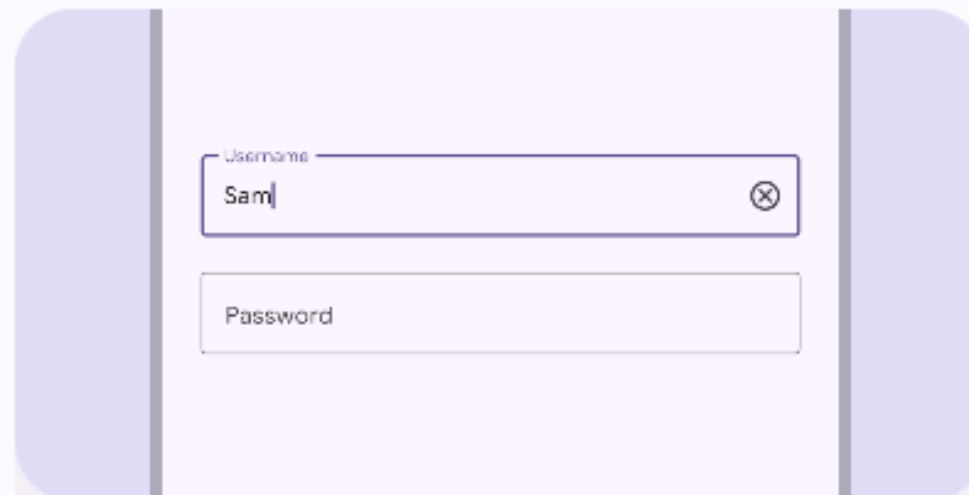
Switch

Switches toggle the selection of an item on and off



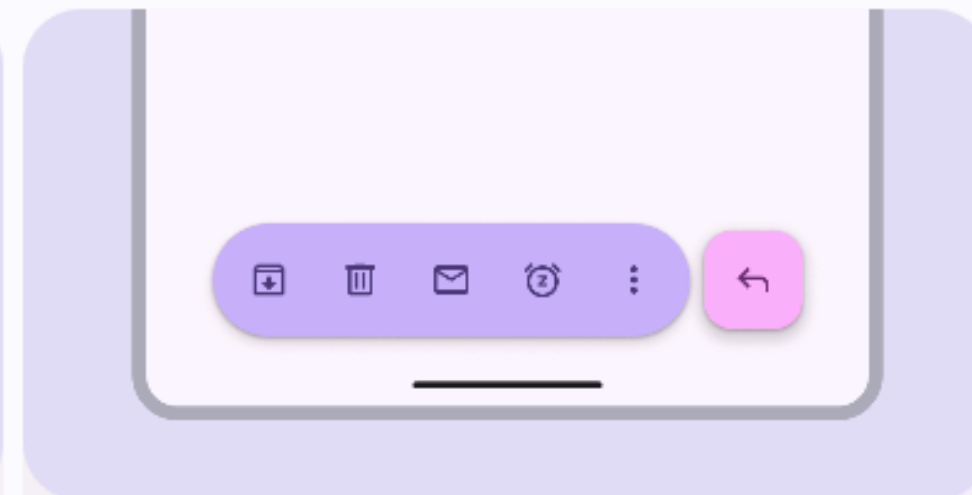
Tabs

Tabs organize content across different screens and views



Text fields

Text fields let users enter text into a UI



Toolbars

Toolbars display frequently used actions relevant to the current page

Add to list

New settings available

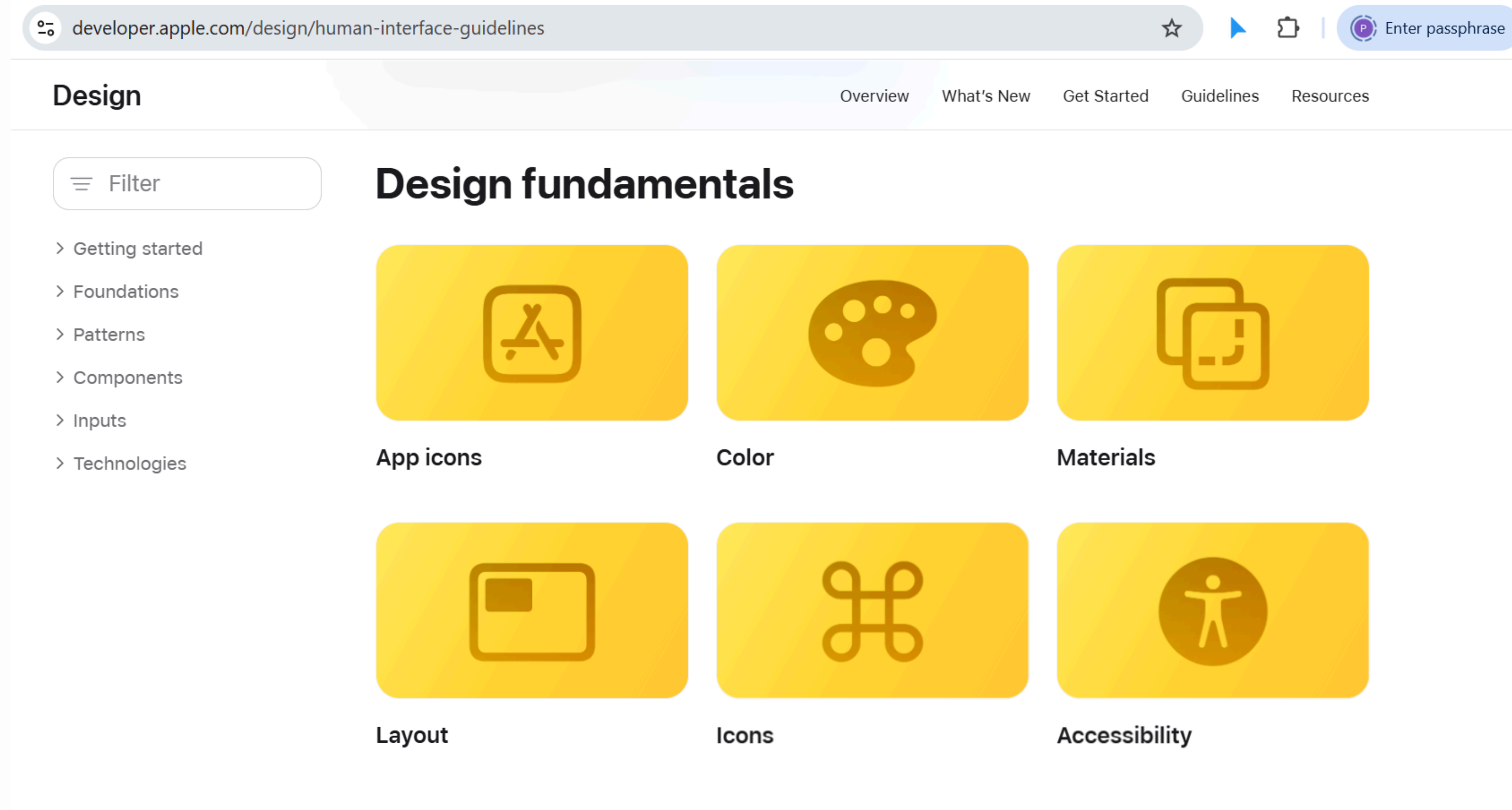
Now you can adjust the uploaded image quality, and upgrade your available storage space.

[Learn more](#)

Tooltips

Tooltips display brief labels or messages

- **Human Interface Guidelines**



<https://developer.apple.com/design/human-interface-guidelines>

- **เว็บไซต์ที่เกี่ยวข้อง**

- <https://www.figma.com/>
- <https://tailwindcss.com/>
- <https://flowbite.com/>
- <https://daisyui.com/>
- <https://mui.com/material-ui/>
- <https://getbootstrap.com/>
- <https://www.flutterflow.io/>
- <https://docs.flutter.dev/ui/widgets>

2. Flutter Widgets

ตาราง 2-1 : สรุป Flutter Widgets ทั้ง 27 ตัว

Item	Flutter Widget	คำอธิบาย / การใช้งาน
1.	AppBar	ใช้เป็นแถบด้านบนของหน้าจอแอปฯ เพื่อแสดง ชื่อกำหนดหน้าแอปฯ ปุ่มค้นหา หรือปุ่มตั้งค่า
2.	Avatar (CircleAvatar)	ใช้แสดงรูปโปรไฟล์แบบวงกลม หรืออักษรย่อ เช่น รูปโปรไฟล์ในหน้าบัญชีผู้ใช้ หรือ ในรายชื่อแชต
3.	Badge	ใช้แสดงจำนวนแจ้งเตือนหรือสถานะเล็ก ๆ บนไอคอน เช่น จำนวนสินค้าบนไอคอนตะกร้า จำนวนการแจ้งเตือนบนไอคอนกระดิ่ง
4.	Button (Elevated)	ใช้เป็นปุ่มสำหรับกดสั่งการ เช่น ปุ่ม “เข้าสู่ระบบ”, “บันทึก”, “ยืนยัน”
5.	Card	ใช้แสดงกรอบที่มีเงาและมุมโค้ง กรอบข้อมูลหรือรูปภาพ เช่น กรอบกรอบข้อมูลสินค้าในหน้า e-commerce
6.	Carousel (Slider)	ใช้แสดงรูปภาพหรือเนื้อหาแบบสไลด์เลื่อนอัตโนมัติ เช่น โฆษณา, แนะนำเพื่อนใน Social Media (Instagram, Facebook)
7.	Checkbox	ใช้เป็นกล่องให้ผู้ใช้เลือกเปิด/ปิด หรือเลือกหลายตัวเลือก เช่น “ยอมรับเงื่อนไขการใช้บริการ”, “สมัครรับข่าวสาร”
8.	Chip	ใช้แสดงข้อมูลสั้น ๆ เช่น แท็ก (#Flutter), หมวดหมู่สินค้า, สมาชิกกลุ่ม
9.	Date Picker	ใช้แสดงกล่องเลือกวันที่หรือเวลาในแอปฯ เช่น วันที่จองโรงแรม, วันที่นัดหมาย
10.	Dialog	ใช้แสดงกล่องข้อความแจ้งเตือนหรือยืนยัน เช่น “คุณแน่ใจหรือไม่ที่จะลบข้อมูลนี้?”
11.	Divider	ใช้เป็นเส้นแบ่งระหว่างเนื้อหา เช่น คั่นระหว่างรายการใน ListView
12.	Icon	ใช้แสดงสัญลักษณ์ในแอป เช่น home, search, setting
13.	List	ใช้แสดงรายการข้อมูลหลายรายการในแนวตั้ง เช่น รายการสินค้า, รายชื่อผู้ติดต่อ
14.	Loading / Progress	ใช้แสดงสถานะการโหลด เช่น โหลดข้อมูลจาก API
15.	Menu (Popup)	ใช้แสดงเมนูแบบ popup เมื่อกดปุ่ม เช่น เมนู “แก้ไข / ลบ / แชร์”

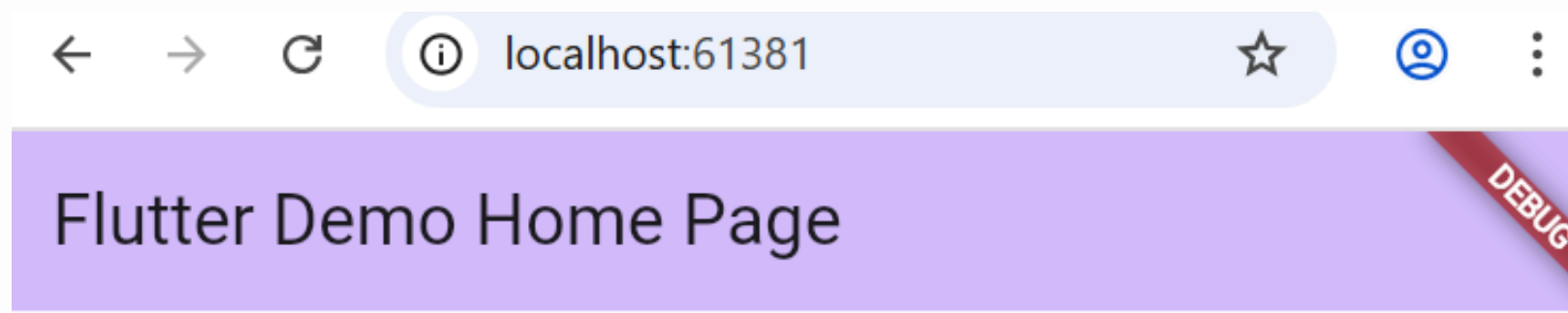
Item	Flutter Widget	คำอธิบาย / การใช้งาน
16.	Navigation (Bottom Nav)	ใช้เป็นเมนูด้านล่างจอสำหรับสลับหน้าในแอป เช่น “Home / Cart / History / Profile”
17.	Radio Button	ใช้เลือกตัวเลือกเพียงอย่างเดียวยจากหลายตัว เช่น เพศ “ชาย / หญิง”
18.	Search (TextField)	ใช้เป็นช่องกรอกข้อความสำหรับค้นหาข้อมูล เช่น ค้นหาสินค้า, ค้นหารายชื่อลูกค้า
19.	Shape (Container)	ใช้สร้างรูปทรงต่าง ๆ เช่น สี่เหลี่ยม วงกลม
20.	Sheet (BottomSheet)	ใช้แสดงแผ่นข้อมูลที่เลื่อนขึ้นมาจากด้านล่างของหน้าจอ เช่น รายละเอียดเพิ่มเติม, เมนูแชร์
21.	Slider	ใช้ปรับค่าตัวเลขโดยการเลื่อน เช่น ปรับระดับเสียง, เลือกช่วงอายุ
22.	Snackbar	ใช้แสดงข้อความแจ้งเตือนชั่วคราว ด้านล่างของหน้าจอ เช่น “บันทึกสำเร็จ”
23.	Switch	ใช้ปิด/เปิดค่าบูลีน เช่น บล็อก/ไม่บล็อกผู้ใช้งาน, ปิด/เปิดเสียงแจ้งเตือน
24.	Tab (TabBar + TabBarView)	ใช้แบ่งเนื้อหาเป็นแท็บ เช่น “เสื้อ / กางเกง / รองเท้า” หรือ “ภาพรวม / รายละเอียด / รีวิว”
25.	TextField	ใช้เป็นช่องให้กรอกข้อมูล เช่น ชื่อผู้ใช้, รหัสผ่าน, อีเมล
26.	Toolbar (AppBar Actions)	ใช้เป็นปุ่มเครื่องมือที่อยู่บน AppBar เช่น ปุ่มค้นหา, ปุ่มตั้งค่า
27.	Tooltip	ใช้แสดงคำอธิบายสั้น ๆ เมื่อผู้ใช้แตะค้างบน widgets ใด ๆ เช่น แตะบนปุ่ม “ตะกร้า” แล้วแสดงคำว่า “เพิ่มสินค้าเข้าไปในตะกร้า”

1. AppBar

```
AppBar(  
  title: Text('Shopdee'),  
  centerTitle: true,  
  backgroundColor: Color(0xFF2F80EC),  
)
```

คำอธิบาย :

ใช้เป็นแถบด้านบนของหน้าจอแอปฯ เพื่อแสดง ชื่อนี้แอปฯ ปุ่มค้นหา หรือปุ่มตั้งค่า



หากต้องการปิด Debug Mode Banner ให้กำหนดค่า ดังนี้

```
return MaterialApp(  
  debugShowCheckedModeBanner: false,
```

2. Avatar (CircleAvatar)

```
CircleAvatar(  
  radius: 30,  
  backgroundImage:  
    AssetImage('assets/profile.jpg'),  
)
```

คำอธิบาย :

ใช้แสดงรูปโปรไฟล์แบบวงกลม หรืออักษรย่อ เช่น รูปโปรไฟล์ในหน้าบัญชีผู้ใช้ หรือ ในรายชื่อแชต

3. Badge

```
Badge(  
  label: Text('3'),  
  child: Icon(Icons.notifications),  
)
```

คำอธิบาย :

ใช้แสดงจำนวนแจ้งเตือนหรือสถานะเล็ก ๆ บนไอคอน เช่น จำนวนสินค้าบนไอคอนตะกร้า จำนวนการแจ้งเตือนบนไอคอนกระดิ่ง

4. Buttons

```
ElevatedButton(  
  onPressed: () {},  
  child: Text('Click Me'),  
)
```

คำอธิบาย :

ใช้เป็นปุ่มสำหรับกดสั่งการ เช่น ปุ่ม “เข้าสู่ระบบ”, “บันทึก”, “ยืนยัน”

ยังมีปุ่มแบบอื่น ๆ เช่น TextButton, OutlinedButton, IconButton, ElevatedButton

5. Card

```
Card(  
  child: ListTile(  
    title: Text('Flutter Widget'),  
    subtitle: Text('This is a card example'),  
  ),  
)
```

คำอธิบาย :

ใช้แสดงกรอบที่มีเงาและมุมโค้ง กรอบข้อมูลหรือรูปภาพ เช่น กรอบกรอบข้อมูลสินค้าในหน้า e-commerce

6. Carousel (use package: carousel_slider)

```
CarouselSlider(  
  items: [  
    Image.asset('assets/img1.jpg'),  
    Image.asset('assets/img2.jpg'),  
    Image.asset('assets/img3.jpg'),  
  ],  
  options: CarouselOptions(autoPlay: true,  
    height: 200),  
)
```

คำอธิบาย :

ใช้แสดงรูปภาพหรือเนื้อหาแบบสไลด์เลื่อนอัตโนมัติ เช่น โฆษณา, แนะนำเพื่อนใน Social Media (Instagram, Facebook)

7. Checkbox

```
Checkbox(  
  value: true,  
  onChanged: (bool? value) {},  
)
```

คำอธิบาย :

ใช้เป็นกล่องให้ผู้เลือกใช้เปิด/ปิด หรือเลือกหลายตัวเลือก เช่น “ยอมรับเงื่อนไขการใช้บริการ”, “สมัครรับข่าวสาร”

8. Chip

```
Chip(  
  label: Text('Flutter'),  
  avatar: CircleAvatar(child: Text('F')),  
)
```

คำอธิบาย :

ใช้แสดงข้อมูลสั้น ๆ เช่น แท็ก (#Flutter), หมวดหมู่สินค้า, สมาชิกกลุ่ม

9. Date & Time Picker

```
ElevatedButton(  
  onPressed: () async {  
    DateTime? date = await showDatePicker(  
      context: context,  
      initialDate: DateTime.now(),  
      firstDate: DateTime(2000),  
      lastDate: DateTime(2100),  
    );  
  },  
  child: Text('Select Date'),  
)
```

คำอธิบาย :

ใช้แสดงกล่องเลือกวันที่หรือเวลาในแอปฯ เช่น วันที่จองโรงแรม, วันที่นัดหมาย

10. Dialog

```
ElevatedButton(  
  onPressed: () async {  
    bool result = await showDialog<bool>(  
      context: context,  
      builder: (context) => AlertDialog(  
        title: const Text('Confirm'),  
        content: const Text('Do you want to exit?'),  
        actions: [  
          TextButton(  
            onPressed: () => Navigator.pop(context, false),  
            child: const Text('Cancel'),  
          ),  
          TextButton(  
            onPressed: () => Navigator.pop(context, true),  
            child: const Text('OK'),  
          ),  
        ],  
      ),  
    ) ??  
    false; // returns false if dialog dismissed  
  
    print('User chose: $result');  
  },  
  child: const Text('Click Me'),  
)
```

คำอธิบาย :

ใช้แสดงกล่องข้อความแจ้งเตือนหรือยืนยัน เช่น “คุณแน่ใจหรือไม่ที่จะลบข้อมูลนี้?”

11. Divider

```
Divider(color: Colors.grey, thickness: 1)
```

คำอธิบาย :

ใช้เป็นเส้นแบ่งระหว่างเนื้อหา เช่น คั่นระหว่างรายการใน ListView

12. Icon

```
Icon(Icons.home, color: Colors.blue, size: 40)
```

คำอธิบาย :

ใช้แสดงสัญลักษณ์ในแอป เช่น home, search, setting

13. List

```
ListView(  
  children: [  
    ListTile(title: Text('Item 1')),  
    ListTile(title: Text('Item 2')),  
  ],  
)
```

คำอธิบาย :

ใช้แสดงรายการข้อมูลหลายรายการในแนวตั้ง เช่น รายการสินค้า, รายชื่อผู้ติดต่อ

14. Loading & Progress

CircularProgressIndicator()

คำอธิบาย :

ใช้แสดงสถานะการโหลด เช่น โหลดข้อมูลจาก API

15. Menu (PopupMenuButton)

```
PopupMenuButton<String>(
  onSelected: (value) {},
  itemBuilder: (context) => [
    PopupMenuItem(value: 'Edit', child: Text('Edit')),
    PopupMenuItem(value: 'Delete', child: Text('Delete')),
  ],
)
```

คำอธิบาย :

ใช้แสดงเมนูแบบ popup เมื่อกดปุ่ม เช่น เมนู “แก้ไข / ลบ / แชร”

16. Navigation (BottomNavigationBar)

```
bottomNavigationBar: BottomNavigationBar(  
  items: [  
    BottomNavigationBarItem(icon: Icon(Icons.home), label: 'Home'),  
    BottomNavigationBarItem(icon: Icon(Icons.person), label: 'Profile'),  
  ],  
)
```

คำอธิบาย :

ใช้เป็นเมนูด้านล่างจอสำหรับสลับหน้าในแอป เช่น “Home / Cart / History / Profile”

17. Radio Button

```
int? _selected = 1; // 1 = ชาย, 2 = หญิง

RadioGroup<int>(
  groupValue: _selected,
  onChanged: (int? value) {
    setState(() {
      _selected = value;
      print('Selected gender: $_selected');
    });
  },

  child: Row(
    mainAxisAlignment: MainAxisAlignment.center,
    children: <Widget>[
      Text('ชาย'),
      Radio<int>(
        value: 1,

      ),

      Text('หญิง'),
      Radio<int>(
        value: 2,
      ),

    ],
  ),
)
```

คำอธิบาย :

ใช้เลือกตัวเลือกเพียงอย่างเดียวจากหลายตัว เช่น เพศ “ชาย / หญิง”

18. Search (TextField + Icon)

```
TextField(  
  decoration: InputDecoration(  
    prefixIcon: Icon(Icons.search),  
    hintText: 'Search...!',  
    border: OutlineInputBorder(),  
  ),  
)
```

คำอธิบาย :

ใช้เป็นช่องกรอกข้อความสำหรับค้นหาข้อมูล เช่น ค้นหาสินค้า, ค้นหารายชื่อผู้ใช้

ตัวอย่างช่องค้นหาสินค้า บน AppBar

🔍 ค้นหาเสื้อผ้า...

```
appBar: AppBar(  
  centerTitle: true,  
  backgroundColor: Color(0xFF2F80EC),  
  title: SizedBox(  
    height: 40,  
    child: TextField(  
      decoration: InputDecoration(  
        hintText: 'ค้นหาเสื้อผ้า...',  
        filled: true,  
        fillColor: Colors.white,  
        contentPadding: const EdgeInsets.symmetric(vertical: 0, horizontal: 16),  
        border: OutlineInputBorder(  
          borderRadius: BorderRadius.circular(20),  
          borderSide: BorderSide.none,  
        ), // OutlineInputBorder  
        prefixIcon: const Icon(Icons.search),  
      ), // InputDecoration  
      onChanged: (value) {  
        // Action when a user is keying  
        print('User is typing: $value');  
      },  
    ), // TextField  
  ), // SizedBox  
), // AppBar
```

19. Shape (Container + BoxDecoration)

```
Container(  
  width: 100,  
  height: 100,  
  decoration: BoxDecoration(  
    color: Colors.blue,  
    shape: BoxShape.circle,  
  ),  
)
```

คำอธิบาย :

ใช้สร้างรูปทรงต่าง ๆ เช่น สี่เหลี่ยม วงกลม

20. Sheet (BottomSheet)

```
ElevatedButton(  
  onPressed: () {  
    showModalBottomSheet(  
      context: context,  
      builder: (context) => Container(  
        height: 200,  
        child: Center(child: Text('This is a bottom sheet')),  
      ),  
    );  
  },  
  
  child: Text('Show Botom Sheet'),  
)
```

คำอธิบาย :

ใช้แสดงแผ่นข้อมูลที่เลื่อนขึ้นมาจากด้านล่างของหน้าจอ เช่น รายละเอียดเพิ่มเติม, เมนูแชร์

21. Slider

```
double _sliderValue = 75;

Slider(
  value: _sliderValue,
  min: 0,
  max: 100,
  onChanged: (value) {
    setState(() {
      _sliderValue = value;
      print('Slider value: $_sliderValue');
    });
  },
)
```

คำอธิบาย :

ใช้ปรับค่าตัวเลขโดยการเลื่อน เช่น ปรับระดับเสียง, เลือกช่วงอายุ

22. Snackbar

```
ElevatedButton(  
  onPressed: () {  
    ScaffoldMessenger.of(context).showSnackBar(  
      SnackBar(  
        content: Text('Saved successfully!'),  
        duration: Duration(seconds: 2),  
      ),  
    );  
  },  
  
  child: Text('Show SnackBar'),  
)
```

คำอธิบาย :

ใช้แสดงข้อความแจ้งเตือนชั่วคราว ด้านล่างของหน้าจอ เช่น “บันทึกสำเร็จ”

23. Switch

```
bool _isOn = true;

Switch(
  value: _isOn,
  onChanged: (value) {
    setState(() {
      _isOn = value;
      print('Switch is now: $_isOn');
    });
  },
)
```

คำอธิบาย :

ใช้ปิด/เปิดค่าบูลีน เช่น บล็อก/ไม่บล็อกผู้ใช้งาน, ปิด/เปิดเสียงแจ้งเตือน

24. Tab

```
DefaultTabController(  
  length: 3,  
  child: Scaffold(  
    appBar: AppBar(  
      title: TabBar(tabs: [  
        Tab(text: 'Home'),  
        Tab(text: 'Profile'),  
        Tab(text: 'Settings'),  
      ]),  
    ),  
    body: TabBarView(children: [  
      Center(child: Text('Home Tab')),  
      Center(child: Text('Profile Tab')),  
      Center(child: Text('Settings Tab')),  
    ]),  
  ),  
);
```

คำอธิบาย :

ใช้แบ่งเนื้อหาเป็นแท็บ เช่น “เสื้อ / กางเกง / รองเท้า” หรือ “ภาพรวม / รายละเอียด / รีวิว”

25. Text Field

```
TextField(  
  decoration: InputDecoration(  
    labelText: 'Username',  
    border: OutlineInputBorder(),  
  ),  
)
```

คำอธิบาย :

ใช้เป็นช่องให้กรอกข้อมูล เช่น ชื่อผู้ใช้, รหัสผ่าน, อีเมล

26. Toolbars (AppBar with actions)

```
AppBar(  
  title: Text('Toolbar'),  
  actions: [  
    IconButton(onPressed: () {}, icon: Icon(Icons.search)),  
    IconButton(onPressed: () {}, icon: Icon(Icons.settings)),  
  ],  
)
```

คำอธิบาย :

ใช้เป็นปุ่มเครื่องมือที่อยู่บน AppBar เช่น ปุ่มค้นหา, ปุ่มตั้งค่า

27. Tooltip

```
Tooltip(  
  message: 'This is an info icon',  
  child: Icon(Icons.info),  
)
```

คำอธิบาย :

ใช้แสดงคำอธิบายสั้น ๆ เมื่อผู้ใช้แตะค้างบน widgets ใด ๆ เช่น แตะบนปุ่ม “ตะกร้า” แล้วแสดงคำว่า “เพิ่มสินค้าเข้าไปในตะกร้า”

3. การเปลี่ยนชื่อและไอคอนของ Launcher

- **เปลี่ยน launcher icon (Android/iOS/Web)**

1. นำรูป logo ไปวางในโฟลเดอร์รูปภาพ ดังนี้ C:\shopdee\frontend\mobile\assets\images\logo.png
2. แก้ไขไฟล์ C:\shopdee\frontend\mobile\pubspec.yaml ดังนี้

```
dev_dependencies:  
  flutter_launcher_icons: ^0.14.4  
  
flutter_launcher_icons:  
  android: true  
  ios: false  
  web:  
    generate: true  
  image_path: "assets/images/logo.png"
```

รันคำสั่ง flutter pub get เฉพาะกรณีที่ VSCode ไม่ทำการรันให้อัตโนมัติ

3. รันคำสั่ง

```
C:\shopdee\frontend\mobile> flutter pub run flutter_launcher_icons
```

- **เปลี่ยน launcher name (Android)**

แก้ไขไฟล์ C:\shopdee\frontend\mobile\android\app\src\main\AndroidManifest.xml ดังนี้

```
<application  
    android:label="Your Launcher Name"
```

- **เปลี่ยน launcher name (iOS)**

แก้ไขไฟล์ C:\shopdee\frontend\mobile\ios\Runner\Info.plist ดังนี้

```
<dict>  
    ...  
    <key>CFBundleDisplayName</key>  
    <string>Your Launcher Name</string>
```

ไฟล์รูปภาพ : <https://drive.google.com/drive/folders/1bbHioH4aqq6HteVWg0q5WfTy8vLuNPqr?usp=sharing>

4. Flutter Libraries

ตาราง 4-1 : Flutter Libraries

Item	Library	หน้าที่หลัก	ตัวอย่างการใช้งาน
1	flutter/material.dart	สร้าง UI	Scaffold, Button, Text
2	http	เรียก API	http.post(), http.get()
3	jwt_decoder	ถอดรหัส token	JwtDecoder.decode(token)
4	shared_preferences	เก็บข้อมูลถาวร	prefs.setString('token', ...)
5	intl	จัดการวันที่/เวลา	DateFormat('dd-MM-yyyy')
6	image_picker	เลือกรูป/วิดีโอ	picker.pickImage()

1. import 'package:flutter/material.dart';

```
MaterialApp(  
  title: 'Flutter Demo',  
  theme: ThemeData(  
    colorScheme: ColorScheme.fromSeed(  
      seedColor: const Color.fromARGB(255, 230, 19, 19)),  
    ),  
  );
```

คำอธิบาย :

- คือ Flutter UI framework หลัก ที่ใช้สร้าง Widgets ต่าง ๆ ตามแนวทางของ Material Design เช่น Scaffold, AppBar, TextField, Button, Radio Button ฯลฯ ซึ่งประกอบขึ้นเป็น UI ของแอป
- Widgets ต่าง ๆ เหล่านี้ จะทำงานภายใต้ MaterialApp

2. import 'package:http/http.dart' as http;

```
final response = await http.post(
  Uri.parse('http://localhost:4000/api/login'),
  headers: {'Content-Type': 'application/json'},
  body: jsonEncode({'username': username, 'password': password}),
);
```

```
final response = await http.get(
  Uri.parse('http://localhost:4000/api/profile/$custID'),
  headers: <String, String>{
    'Authorization': 'Bearer $token',
  },
);
```

```
final response = await http.put(
  Uri.parse('http://localhost:4000/api/customer/$custID'),
  headers: <String, String>{
    'Authorization': 'Bearer $token',
  },
);
```

ตัวอย่าง API : <https://dummyjson.com/docs/auth>

คำอธิบาย :

- ใช้สำหรับ เรียก API ผ่าน HTTP เช่น POST, GET, PUT, DELETE
- as http หมายถึงใช้ชื่อย่อ http ตอนเรียก

3. import 'package:jwt_decoder/jwt_decoder.dart';

// ถอดรหัสข้อมูล

```
Map<String, dynamic> decoded = JwtDecoder.decode(token);
```

// ตรวจสอบว่า token หมดอายุหรือยัง

```
bool isExpired = JwtDecoder.isExpired(token);
```

คำอธิบาย :

- ใช้สำหรับ ถอดรหัส (decode) JWT (JSON Web Token)
- สามารถดูข้อมูลภายใน token ได้ เช่น userID, email, expire time

4. import 'package:shared_preferences/shared_preferences.dart';

```
//รับข้อมูลจาก Shared Preference  
SharedPreferences prefs = SharedPreferences.getInstance();  
prefs.setString('token', data['token']);  
prefs.setInt('custID', data['custID']);
```

```
//ดึงข้อมูลจาก Shared Preference  
SharedPreferences prefs = SharedPreferences.getInstance();  
final String? token = prefs.getString('token');  
final int? custID = prefs.getInt('custID');
```

คำอธิบาย :

- ใช้เก็บ ข้อมูลขนาดเล็กแบบ local storage บนอุปกรณ์ เช่น token, user id, username

5. import 'package:intl/intl.dart';

```
String date = DateFormat('yyyy-MM-dd').format(DateTime.now());
```

คำอธิบาย :

- ใช้สำหรับ จัดการรูปแบบวันที่ / เวลา / ตัวเลข / สกุลเงิน

6. `import 'package:image_picker/image_picker.dart';`

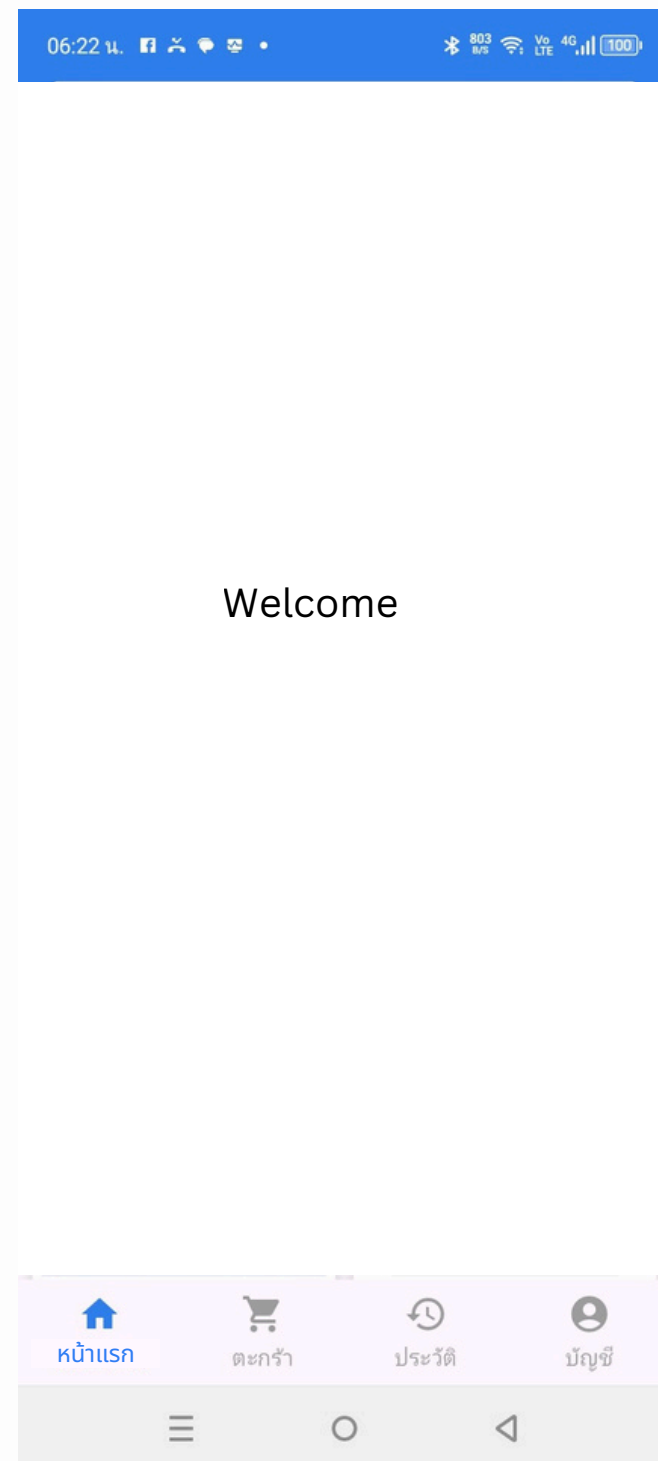
```
final picker = ImagePicker();  
final image = await picker.pickImage(source: ImageSource.gallery);
```

คำอธิบาย :

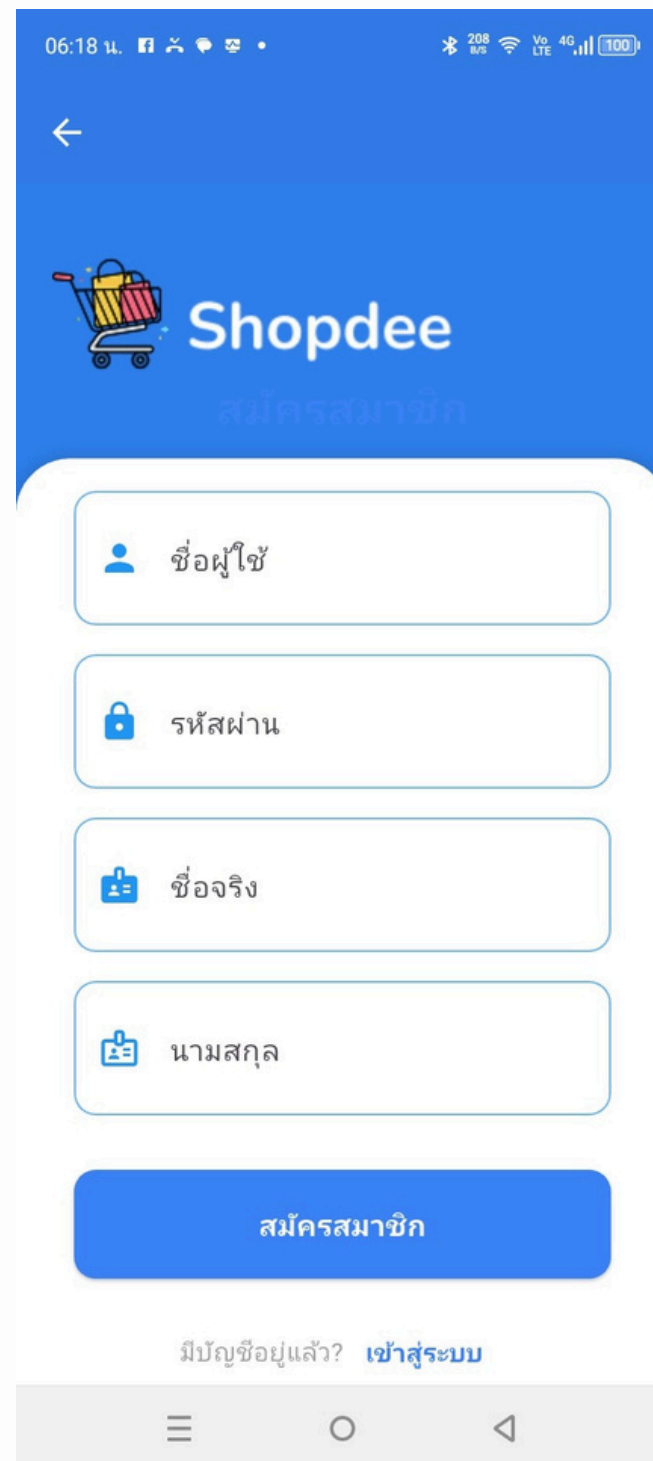
- ใช้เปิด กล้องหรือแกลเลอรี เพื่อเลือกรูปภาพหรือวิดีโอ

5. ตัวอย่างการออกแบบระบบซื้อขายออนไลน์

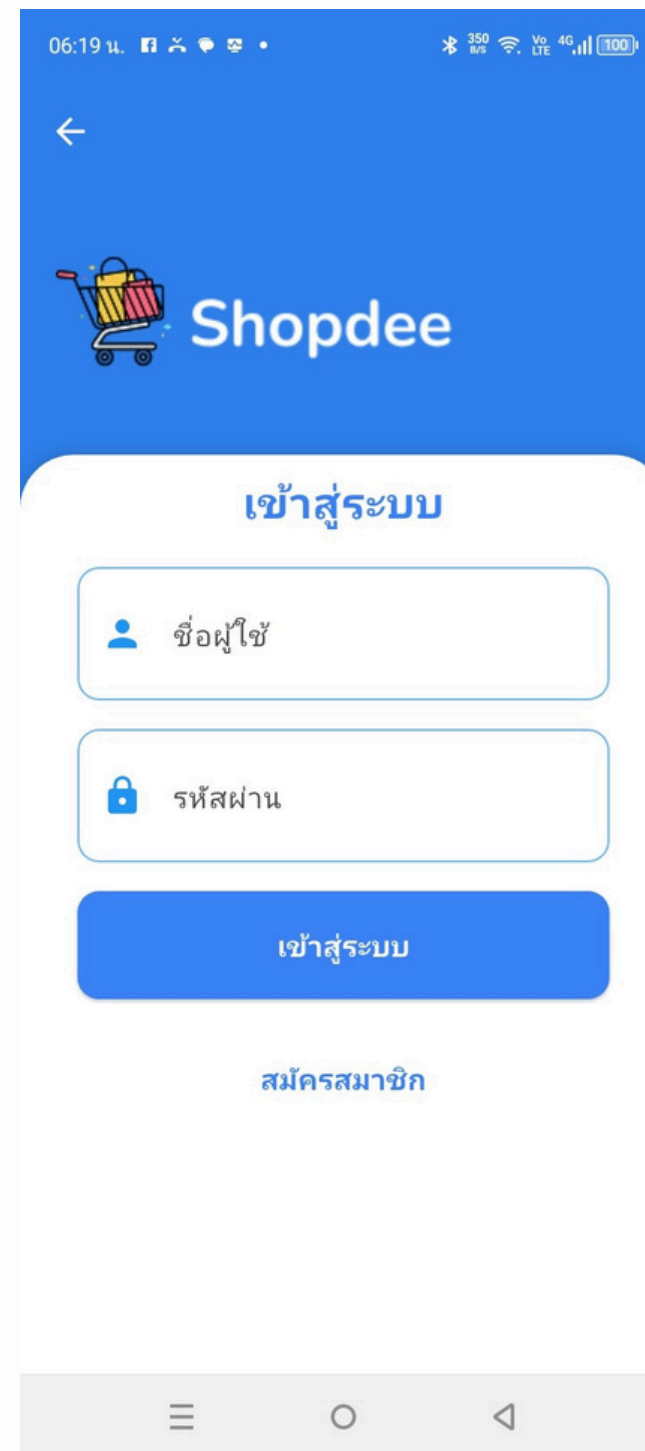
หน้าหลัก



สมัครสมาชิก



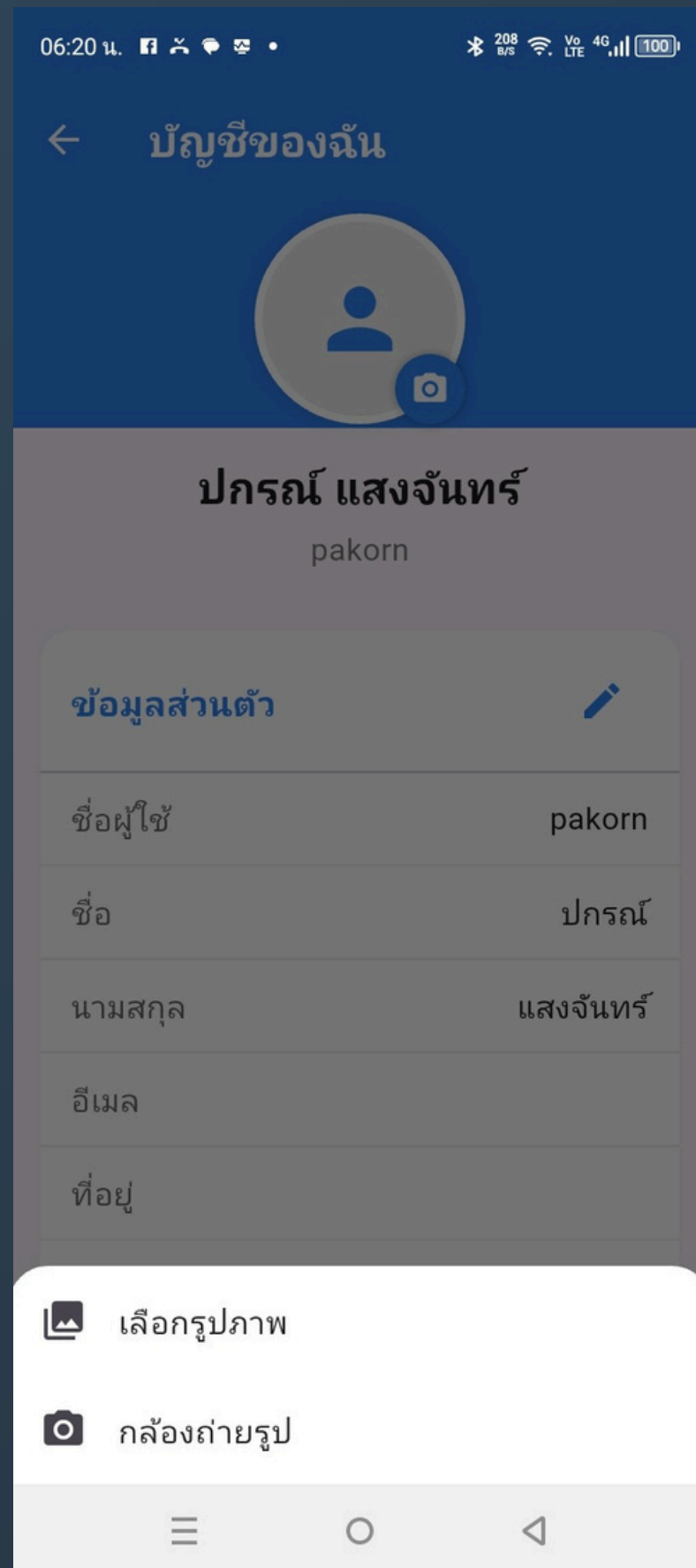
เข้าสู่ระบบ



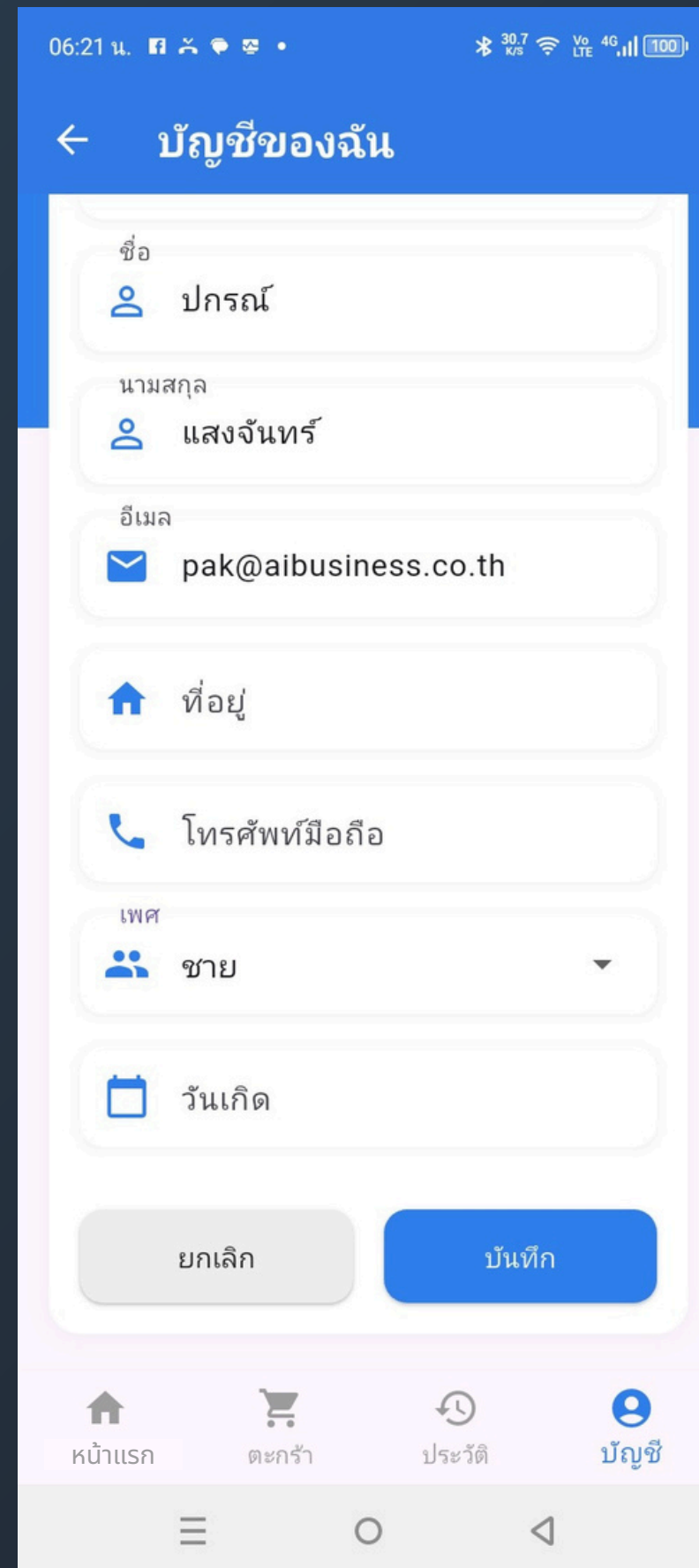
โปรไฟล์



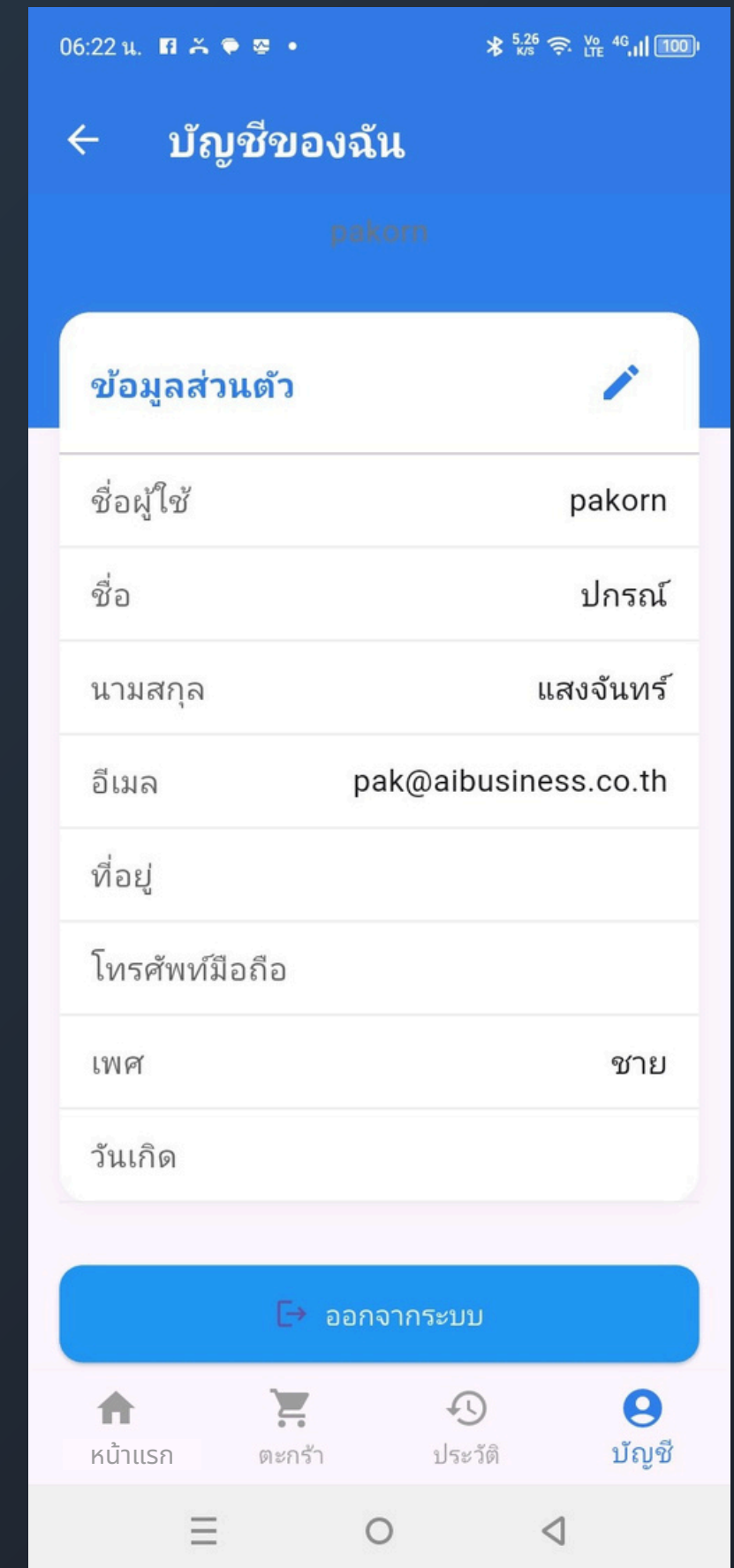
แก้ไขรูปโปรไฟล์



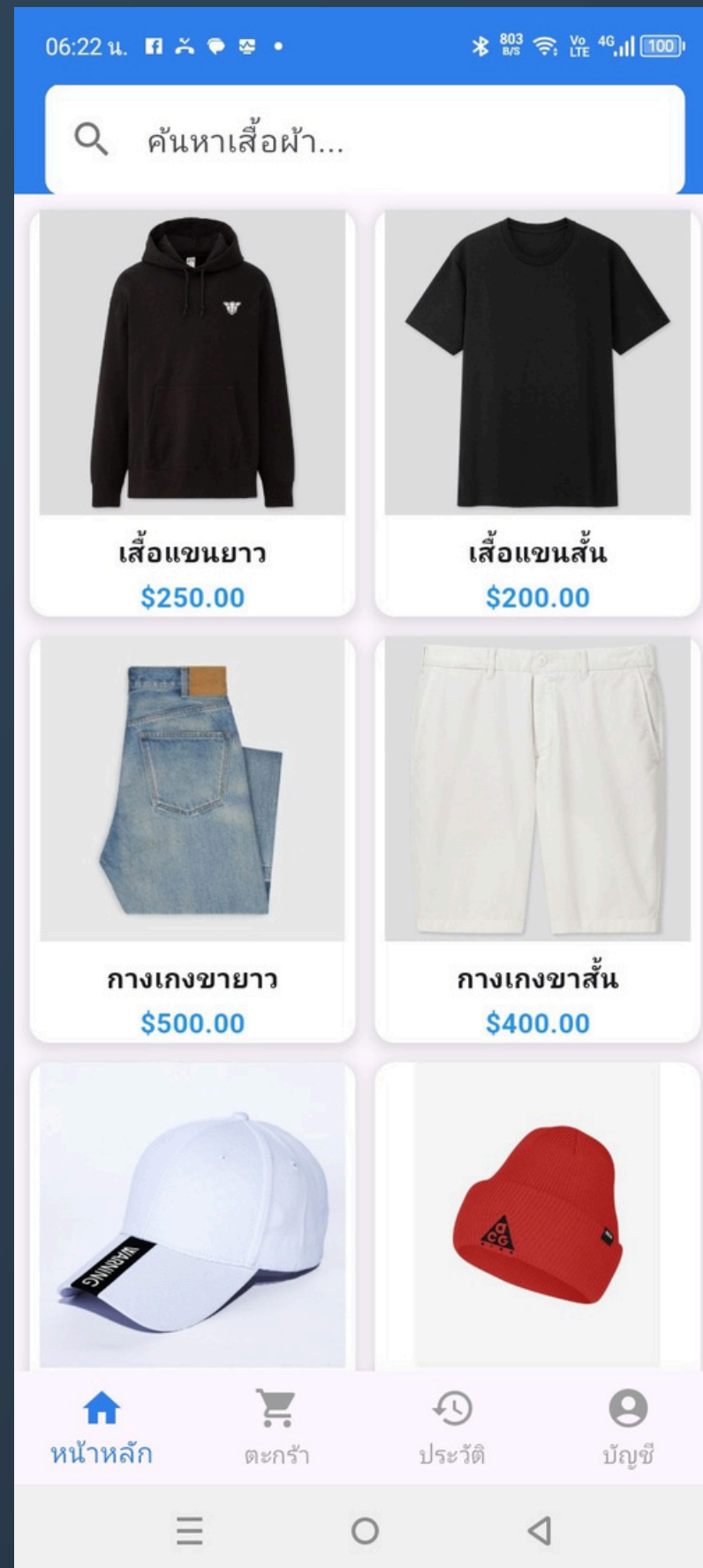
แก้ไขข้อมูลโปรไฟล์



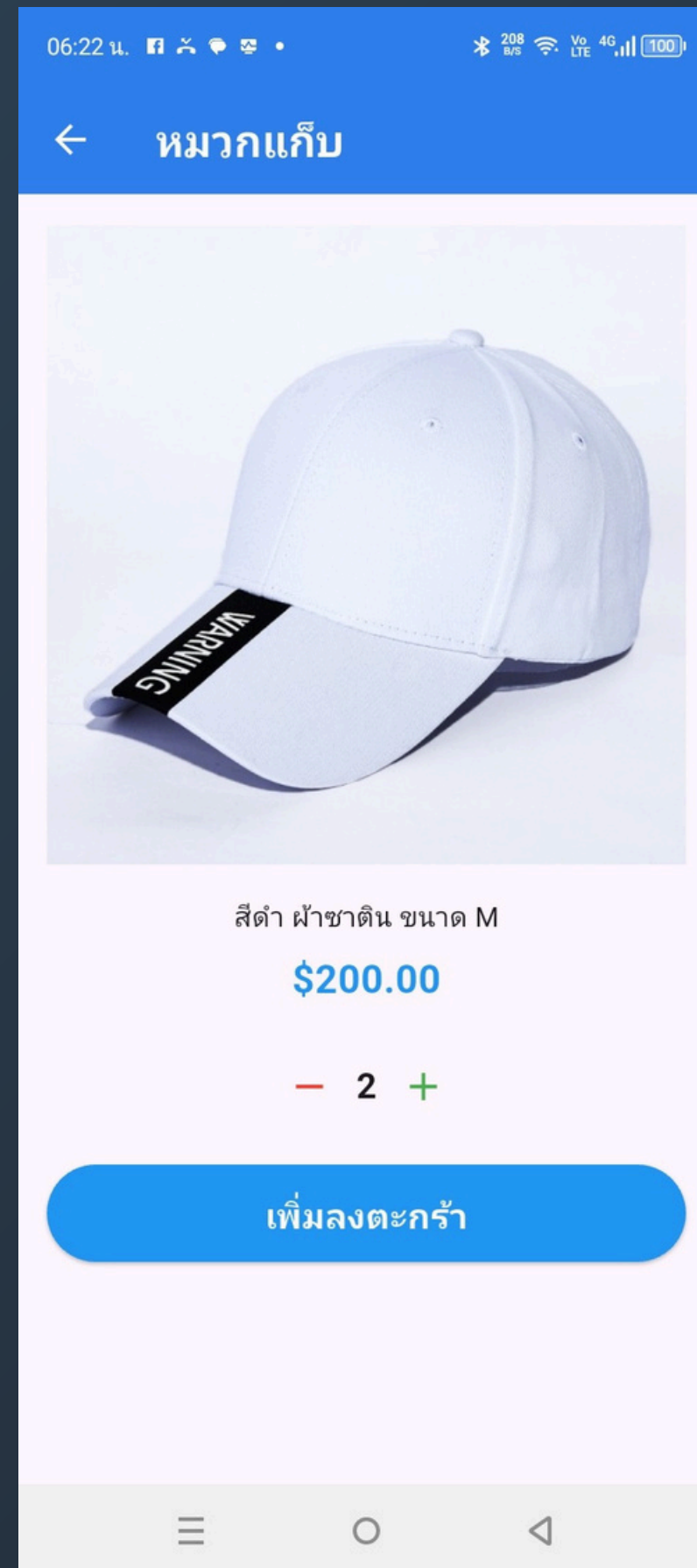
ออกจากระบบ



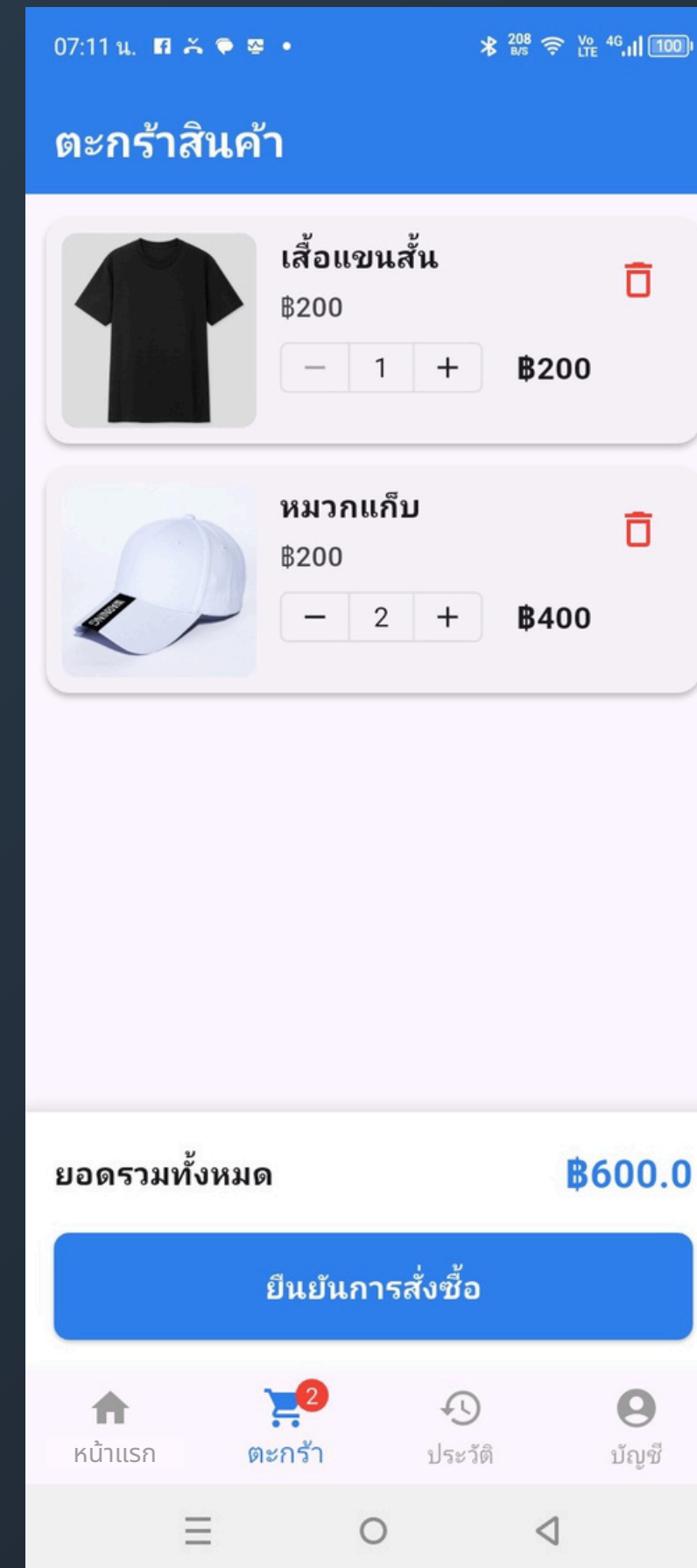
หน้าแรก



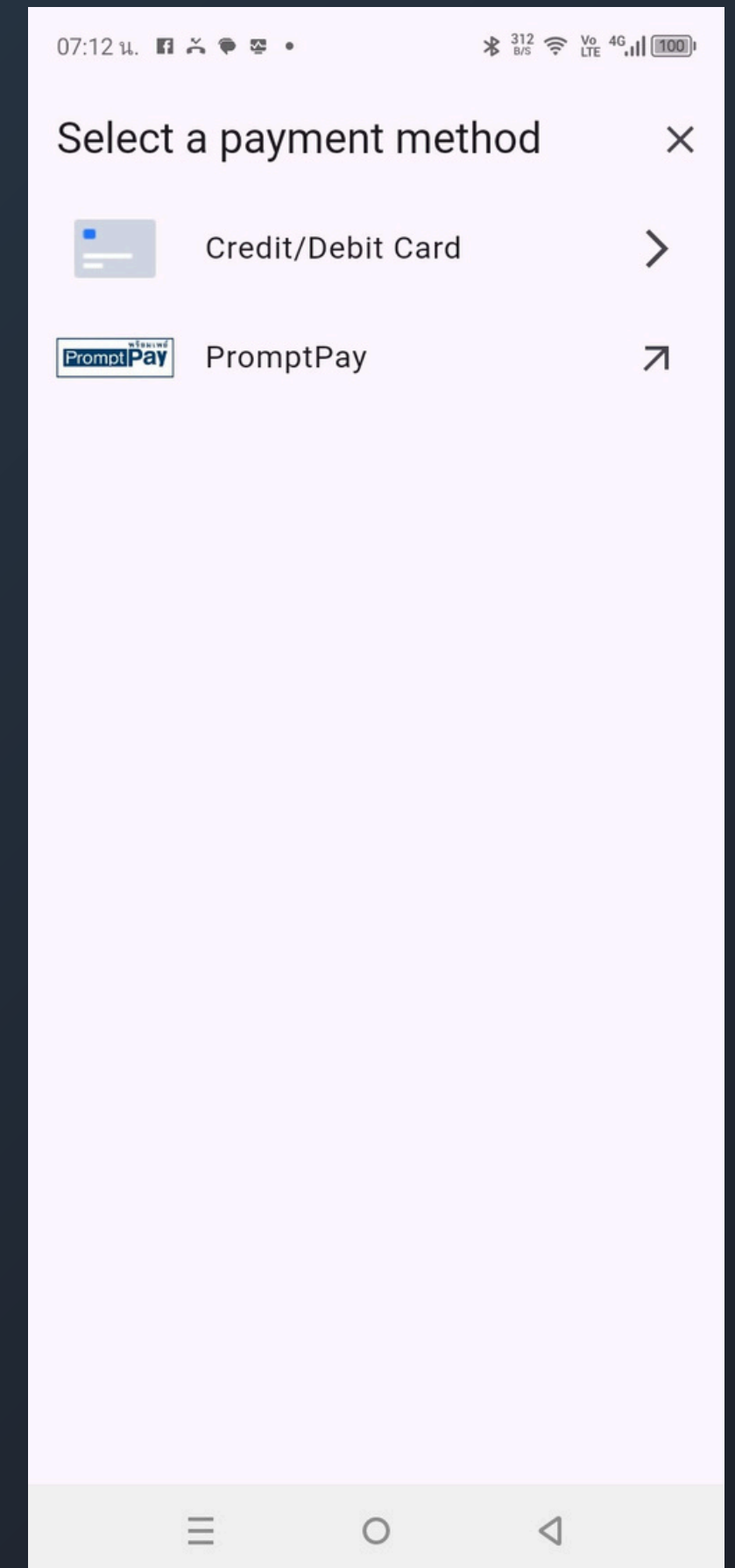
รายละเอียดสินค้า



ตะกร้าสินค้า



ชำระเงิน



ชำระเงิน

07:12 น. 208 B/s Vo LTE 4G 100

← Credit Card

Card Number

Name on card

Expiry Date Security code

MM/YY

Country or region

Thailand

Pay

07:16 น. 4.30 K/s Vo LTE 4G 100

← Credit Card

Card Number

5555555555554444

Name on card

pakorn saengchun

Expiry Date Security code

12/25

...

Country or region

Thailand

Pay

ประวัติการสั่งซื้อ

07:18 น. 3.77 K/s Vo LTE 4G 100

← รายละเอียดคำสั่งซื้อ

คำสั่งซื้อ #15

วันที่สั่งซื้อ: 2025-06-01

สถานะ: ชำระเงินแล้ว

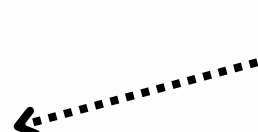
จำนวนสินค้า: 3

ราคารวม: ฿600

รายการสินค้า

เสื้อแขนสั้น	฿200
จำนวน: 1 x ฿200	
หมวกแก๊ป	฿400
จำนวน: 2 x ฿200	

6. หน้าหลัก

1. เริ่มสร้างโปรเจกต์
flutter create **hello_app**  ระบุเครื่องหมายจุด . แทนชื่อโปรเจกต์ หากไม่ต้องการให้สร้าง folder ใหม่

2. รัน Mobile App (หากมีการตั้งชื่อโปรเจกต์ให้ใช้คำสั่ง cd **hello-app**)
flutter run

หมายเหตุ : - หากเชื่อมต่อมือถือหรือเปิด Emulator ระบบจะทำการเลือก device ให้อัตโนมัติ
- หากไม่ได้เชื่อมต่ออะไร ระบบจะให้เลือกว่าจะรัน win app. หรือ web app.

3. ตรวจสอบโครงสร้างของไฟล์ /lib/main.dart

```
1  // ส่วนที่ 1: การ import libraries และไฟล์ที่เกี่ยวข้อง
2  import 'package:flutter/material.dart';
3
4  // ส่วนที่ 2: Main function คือ จุดเริ่มต้นของแอป
   Run | Debug | Profile
5  void main() {
6    runApp(const MyApp());
7  }
8
9  // ส่วนที่ 3: MyApp คือ StatelessWidget ที่ไม่ต้องเปลี่ยนแปลงค่าหรือสถานะในระหว่างที่แอปทำงาน
10 class MyApp extends StatelessWidget {
11   const MyApp({super.key});
12
13   //ทำการสร้างและแสดง widgets ของแอปพลิเคชัน (คล้ายกับ return ใน react)
14   @override
15 > Widget build(BuildContext context) { ...
25 }
26
27 // ส่วนที่ 4: MainPage คือ StatefulWidget ซึ่งสามารถเปลี่ยนแปลงค่าหรือสถานะได้
28 > class MyHomePage extends StatefulWidget { ...
36
37 // ส่วนที่ 5: _MainPageState คือ state ของ MainPage
38 > class _MyHomePageState extends State<MyHomePage> { ...
80
```

3. สร้างไฟล์สำหรับแต่ละเมนูด้านล่าง

3.1 สร้างไฟล์สำหรับเมนูหน้าแรก ไว้ที่ /lib/src/home.dart

```
1  import 'package:flutter/material.dart';
2
3  class HomePage extends StatefulWidget {
4    const HomePage({Key? key}) : super(key: key);
5
6    @override
7    State<HomePage> createState() => _HomePageState();
8  }
9
10 class _HomePageState extends State<HomePage> {
11
12   @override
13   Widget build(BuildContext context) {
14     return Scaffold(
15       appBar: AppBar(
16         title: const Text('หน้าแรก'),
17       ), // AppBar
18       body: Center(
19         child: Column(
20           mainAxisAlignment: MainAxisAlignment.center,
21           children: [
22             const Text(''),
23           ],
24         ), // Column
25       ), // Center
26     ); // Scaffold
27   }
28 }
```

3.2 สร้างไฟล์สำหรับเมนูตะกร้า ไว้ที่ /lib/src/cart.dart

```
1  import 'package:flutter/material.dart';
2
3  class CartPage extends StatefulWidget {
4    const CartPage({Key? key}) : super(key: key);
5
6    @override
7    State<CartPage> createState() => _CartPageState();
8  }
9
10 class _CartPageState extends State<CartPage> {
11
12   @override
13   Widget build(BuildContext context) {
14     return Scaffold(
15       appBar: AppBar(
16         title: const Text('ตะกร้าสินค้า'),
17       ), // AppBar
18       body: Center(
19         child: Column(
20           mainAxisAlignment: MainAxisAlignment.center,
21           children: [
22             const Text(''),
23           ],
24         ), // Column
25       ), // Center
26     ); // Scaffold
27   }
28 }
```

3.3 สร้างไฟล์สำหรับเมนูประวัติ ไว้ที่ /lib/src/history.dart

```
1  import 'package:flutter/material.dart';
2
3  class HistoryPage extends StatefulWidget {
4    const HistoryPage({Key? key}) : super(key: key);
5
6    @override
7    _HistoryPageState createState() => _HistoryPageState();
8  }
9
10 class _HistoryPageState extends State<HistoryPage> {
11
12   @override
13   Widget build(BuildContext context) {
14     return Scaffold(
15       appBar: AppBar(
16         title: const Text('ประวัติการสั่งซื้อ'),
17       ), // AppBar
18       body: Center(
19         child: Column(
20           mainAxisAlignment: MainAxisAlignment.center,
21           children: [
22             const Text(''),
23           ],
24         ), // Column
25       ), // Center
26     ); // Scaffold
27   }
28 }
```

3.4 สร้างไฟล์สำหรับเมนูบัญชี ไว้ที่ /lib/src/account.dart

```
1  import 'package:flutter/material.dart';
2
3  class AccountPage extends StatefulWidget {
4    const AccountPage({Key? key}) : super(key: key);
5
6    @override
7    _AccountPageState createState() => _AccountPageState();
8  }
9
10 class _AccountPageState extends State<AccountPage> {
11
12   @override
13   Widget build(BuildContext context) {
14     return Scaffold(
15       appBar: AppBar(
16         title: const Text('บัญชีผู้ใช้'),
17       ), // AppBar
18       body: Center(
19         child: Column(
20           mainAxisAlignment: MainAxisAlignment.center,
21           children: [
22             const Text(''),
23           ],
24         ), // Column
25       ), // Center
26     ); // Scaffold
27   }
28 }
```

4. แก้ไขไฟล์ /lib/main.dart แต่ละส่วน ดังนี้

4.1 ส่วนที่ 1

```
1  // ส่วนที่ 1: การ import libraries และไฟล์ที่เกี่ยวข้อง
2  import 'package:flutter/material.dart';
3  import 'package:mobile/src/home.dart';
4  import 'package:mobile/src/cart.dart';
5  import 'package:mobile/src/history.dart';
6  import 'package:mobile/src/account.dart';
7
```

4.2 ส่วนที่ 2 (คองเตน)

```
8  // ส่วนที่ 2: Main function คือ จุดเริ่มต้นของแอป
   Run | Debug | Profile
9  void main() {
10 |   runApp(const MyApp());
11 }
12
```

4.3 ส่วนที่ 3

```
13  // ส่วนที่ 3: MyApp คือ StatelessWidget ที่ไม่ต้องเปลี่ยนแปลงค่าหรือสถานะในระหว่างที่แอปทำงาน
14  class MyApp extends StatelessWidget {
15      const MyApp({super.key});
16
17      //ทำการสร้างและแสดง widgets ของแอปพลิเคชัน (คล้ายกับ return ใน react)
18      @override
19      Widget build(BuildContext context) {
20          return MaterialApp(
21              debugShowCheckedModeBanner: false, // ซ่อนแถบ debug
22              title: 'Shopdee', // กำหนดชื่อแอปพลิเคชัน
23              theme: ThemeData(
24                  colorScheme: ColorScheme.fromSeed(seedColor: Colors.blue), // กำหนดสีหลักของแอปพลิเคชัน
25              ), // ThemeData
26              home: const MyHomePage(title: 'ช้อปดี'), // กำหนดหน้าหลักและหัวเรื่องของแอปพลิเคชัน
27          ); // MaterialApp
28      }
29  }
30
```

4.4 ส่วนที่ 4 (คงเดิม)

```
31  // ส่วนที่ 4: MainPage คือ StatefulWidget ซึ่งสามารถเปลี่ยนแปลงค่าหรือสถานะได้
32  class MyHomePage extends StatefulWidget {
33      const MyHomePage({super.key, required this.title});
34
35      final String title;
36
37      @override
38      State<MyHomePage> createState() => _MyHomePageState();
39  }
40
```

4.5 ส่วนที่ 5

```
41 // ส่วนที่ 5: _MainPageState คือ state ของ MainPage
42 class _MyHomePageState extends State<MyHomePage> {
43     int _selectedIndex = 0; // index ของหน้าใน bottom navigation
44
45     // รายการของหน้าในแต่ละ tab
46     final List<Widget> _pages = [
47         const HomePage(),
48         const CartPage(),
49         const HistoryPage(),
50         const AccountPage(),
51     ];
52
53     // เรียกใช้เมื่อผู้ใช้กดที่แต่ละ tab ใน bottom navigation
54     void _onItemTapped(int index) {
55         setState(() {
56             _selectedIndex = index;
57         });
58     }
59
60     // อัปเดต index ของหน้าเมื่อเปลี่ยน tab
61     void setSelectedIndex(int index) {
62         setState(() {
63             _selectedIndex = index;
64         });
65     }
66 }
```

```

67 //ทำการสร้าง widget ของหน้าหลักของแอปพลิเคชัน
68 @override
69 Widget build(BuildContext context) {
70   // Scaffold เป็น widget ที่ให้โครงสร้างพื้นฐานสำหรับ Material Design
71   // แอปพลิเคชัน เช่น AppBar, Body, FloatingActionButton เป็นต้น
72   return Scaffold(
73     body: _pages[_selectedIndex], // แสดงหน้าแต่ละ tab
74     bottomNavigationBar: BottomNavigationBar(
75       type: BottomNavigationBarType.fixed, // แสดง label ทุกอัน
76       items: <BottomNavigationBarItem>[
77         const BottomNavigationBarItem(
78           icon: Icon(Icons.home),
79           label: 'หน้าแรก',
80         ), // BottomNavigationBarItem
81         BottomNavigationBarItem(
82           icon: Icon(Icons.shopping_cart),
83           label: 'ตะกร้า',
84         ), // BottomNavigationBarItem
85         const BottomNavigationBarItem(
86           icon: Icon(Icons.history),
87           label: 'ประวัติ',
88         ), // BottomNavigationBarItem

```

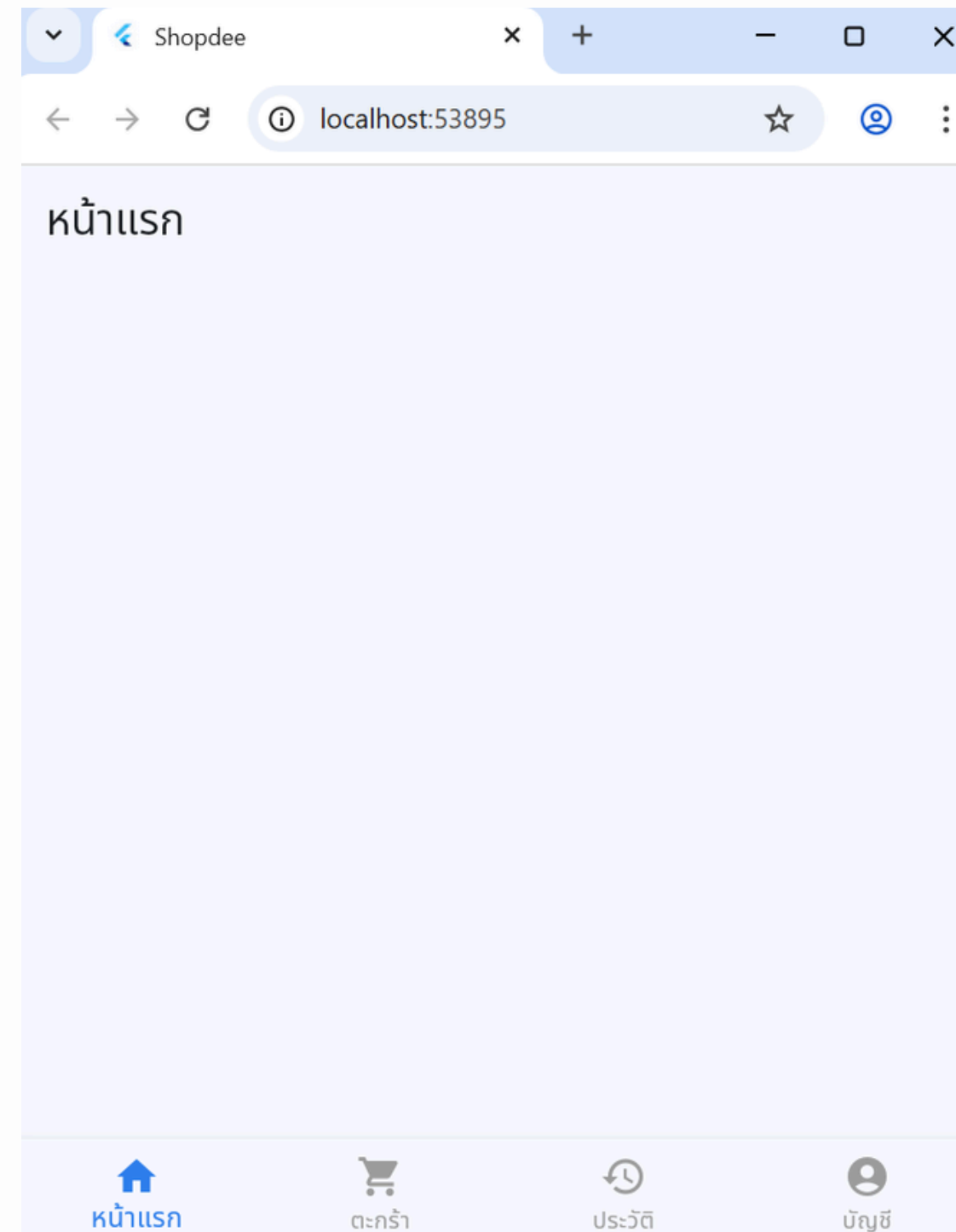
```

89     const BottomNavigationBarItem(
90       icon: Icon(Icons.account_circle),
91       label: 'บัญชี',
92     ), // BottomNavigationBarItem
93   ], // <BottomNavigationBarItem>[]
94   currentIndex: _selectedIndex, // tab ที่ถูกเลือกอยู่
95   selectedItemColor:  const Color(0xFF2F80ED), // สีของ tab ที่ถูกเลือก
96   unselectedItemColor:  Colors.grey, // สีของ tab ที่ไม่ได้เลือก
97   onTap: _onItemTapped, // เมื่อกด tab
98 ), // BottomNavigationBar
99 ); // Scaffold
100 }
101 }

```

5. Restart แอปพลิเคชัน

กด r เพื่อทำการ restart แอปพลิเคชัน



ตาราง 1.1 เปรียบเทียบ Flutter Widget กับ HTML tag

Flutter Widget	HTML Tag	คำอธิบาย
MaterialApp	<html>	ตัว root ของแอป Material Design (โครงสร้างหลัก)
Scaffold	<body>	โครงหน้ามาตรฐานของแอป Material Design
AppBar (ไม่มีในโค้ดนี้ แต่ Scaffold รองรับ)	<header>	ส่วนหัวของหน้า
BottomNavigationBar	<footer> หรือ <nav>	แถบเมนูด้านล่าง
BottomNavigationBarItem	<a>	แต่ละปุ่มในเมนู
Icon(Icons.home)	<svg> หรือ <i class="...">	ไอคอน
Text('หน้าแรก')	 หรือ <p>	ข้อความ label
StatefulWidget (MyHomePage)	<div> + script จัดการ state	องค์ประกอบที่มี state เปลี่ยนได้
StatelessWidget (MyApp)	<div> คงที่	องค์ประกอบที่ไม่มี state
List<Widget> _pages	<section> หรือ <div> หลายตัว	เนื้อหาแต่ละหน้า
HomePage() / CartPage() / HistoryPage() / AccountPage()	<section> หรือ <article>	เนื้อหาของแต่ละแท็บ
setState()	JavaScript DOM update (document.querySelector(...))	อัปเดต UI ตาม state ใหม่

ตาราง 1.2 เปรียบเทียบ Flutter Widget (Property) กับ TailwindCSS (class name)

Flutter (Property / Method)	Tailwind CSS Class	คำอธิบาย
selectedItemColor: Color(0xFF2F80ED)	text-[#2F80ED]	สีตัวอักษร/ไอคอนเมื่อถูกเลือก
unselectedItemColor: Colors.grey	text-gray-500	สีตัวอักษร/ไอคอนเมื่อไม่ได้เลือก
type: BottomNavigationBarType.fixed	flex justify-between	จัดปุ่มคงที่ทั้งหมด
Icon(Icons.home)	w-6 h-6 (หรือขนาดที่ต้องการ)	กำหนดขนาดไอคอน
label: 'หน้าแรก'	text-sm หรือ text-base	ขนาดข้อความ
currentIndex: _selectedIndex	aria-current="page" + font-bold	สถานะปุ่มที่ถูกเลือก
body: _pages[_selectedIndex]	flex-1	ให้เนื้อหาขยายเต็มพื้นที่
BottomNavigationBar ใน Scaffold	fixed bottom-0 w-full border-t	ติดขอบล่างและเต็มความกว้าง
EdgeInsets (ถ้ามี)	p-x, m-x	padding / margin
Color.fromARGB(255,17,6,215)	bg-[#1106D7]	กำหนดสีพื้นหลัง

7. หน้าเข้าสู่ระบบ

โครงสร้างของไฟล์ /lib/src/login.dart

```
1 | // ส่วนที่ 1: การ import libraries และไฟล์ที่เกี่ยวข้อง
2 | import 'package:flutter/material.dart';
3 | import 'package:http/http.dart' as http;
4 | import 'dart:convert';
5 | import 'package:shared_preferences/shared_preferences.dart';
6 |
7 | // ส่วนที่ 4: LoginPage คือ StatefulWidget ซึ่งสามารถเปลี่ยนแปลงค่าหรือสถานะได้
8 | > class LoginPage extends StatefulWidget { ...
15
16 | > // ส่วนที่ 5: _LoginPageState คือ class ที่ใช้จัดเก็บ/จัดการ state ของ LoginPage ...
18 | > class _LoginPageState extends State<LoginPage> {
19 |     // Define text editing controllers to handle user input
20 |     final TextEditingController _usernameController = TextEditingController();
21 |     final TextEditingController _passwordController = TextEditingController();
22 |
23 |     // Login function to send and get user data from API
24 | > Future<void> _login() async { ...
55
56 |     // Render login page UI
57 |     @override
58 | > Widget build(BuildContext context) { ...
92 | }
```

1. แก้ไขไฟล์ /lib/src/login.dart แต่ละส่วน ดังนี้

- ส่วนที่ 1

```
1 | // ส่วนที่ 1: การ import libraries และไฟล์ที่เกี่ยวข้อง
2 | import 'package:flutter/material.dart';
3 | import 'package:http/http.dart' as http;
4 | import 'dart:convert';
5 | import 'package:shared_preferences/shared_preferences.dart';
```

- ส่วนที่ 4 (คงเดิม)

```
6
7 | // ส่วนที่ 4: LoginPage คือ StatefulWidget ซึ่งสามารถเปลี่ยนแปลงค่าหรือสถานะได้
8 | class LoginPage extends StatefulWidget {
9 |   | const LoginPage({Key? key}) : super(key: key); //กำหนด constructor
10
11 |   | @override
12 |   | State<LoginPage> createState() => _LoginPageState(); //สร้าง state object
13 |   |
14 | }
```

- ส่วนที่ 5

```
15
16 | // ส่วนที่ 5: _LoginPageState คือ class ที่ใช้จัดเก็บ/จัดการ state ของ LoginPage
17 | // สร้าง UI (widget) และเก็บค่า state ต่าง ๆ
18 | class _LoginPageState extends State<LoginPage> {
19 |     // Define text editing controllers to handle user input
20 |     final TextEditingController _usernameController = TextEditingController();
21 |     final TextEditingController _passwordController = TextEditingController();
22 |
23 |     // Login function to send and get user data from API
24 |     Future<void> _login() async {
25 |         final email = _usernameController.text;
26 |         final password = _passwordController.text;
27 |
28 |         try {
29 |             // Send and get login data from API
30 |             final response = await http.post(
31 |                 Uri.parse('http://localhost:4000/api/login'),
32 |                 headers: {'Content-Type': 'application/json'},
33 |                 body: jsonEncode({'username': email, 'password': password}),
34 |             );
35 |             final data = jsonDecode(response.body);
```

```

36
37     if (data['status'] == true) {
38         // Save user data to shared preferences
39         final prefs = await SharedPreferences.getInstance();
40         await prefs.setString('token', data['token'] ?? '');
41         await prefs.setString('custID', data['custID'] ?? '');
42         Navigator.pushReplacementNamed(context, '/'); //go to main page
43     } else {
44         // Login failed, show error message
45         ScaffoldMessenger.of(context).showSnackBar(
46             SnackBar(content: Text(data['message'] ?? 'Login failed')),
47         );
48     }
49 } catch (e) {
50     ScaffoldMessenger.of(context).showSnackBar(
51         SnackBar(content: Text('Error: $e')),
52     );
53 }
54 }
55

```

```

56 | // Render login page UI
57 | @override
58 | Widget build(BuildContext context) {
59 |   return Scaffold(
60 |     appBar: AppBar(title: const Text('Login')),
61 |     body: Padding(
62 |       padding: const EdgeInsets.all(16.0),
63 |       child: Column(
64 |         mainAxisAlignment: MainAxisAlignment.center,
65 |         children: [
66 |           TextField(
67 |             controller: _usernameController,
68 |             decoration: const InputDecoration(
69 |               labelText: 'ชื่อผู้ใช้',
70 |               border: OutlineInputBorder(),
71 |             ), // InputDecoration
72 |           ), // TextField
73 |           const SizedBox(height: 16),
74 |           TextField(
75 |             controller: _passwordController,
76 |             decoration: const InputDecoration(
77 |               labelText: 'Password',
78 |               border: OutlineInputBorder(),
79 |             ), // InputDecoration
80 |             obscureText: true,
81 |           ), // TextField

```

```
82         const SizedBox(height: 24),
83         ElevatedButton(
84             onPressed: _login,
85             child: const Text('Login'),
86         ), // ElevatedButton
87     ],
88 ), // Column
89 ), // Padding
90 ); // Scaffold
91 }
92 }
```

8. หน้าโปรไฟล์

โครงสร้างของไฟล์ /lib/src/account.dart

```
1  // ส่วนที่ 1: การ import libraries และไฟล์ที่เกี่ยวข้อง
2  import 'package:flutter/material.dart';
3  import 'package:http/http.dart' as http;
4  import 'package:jwt_decoder/jwt_decoder.dart';
5  import 'dart:convert';
6  import 'package:shared_preferences/shared_preferences.dart';
7  import 'package:intl/intl.dart';
8  import 'package:image_picker/image_picker.dart';
9  import 'login.dart';
10 import 'register.dart';
11 import 'api_config.dart';
12
13 // ส่วนที่ 4: MainPage คือ StatefulWidget ซึ่งสามารถเปลี่ยนแปลงค่าหรือสถานะได้
14 class AccountPage extends StatefulWidget {
15   const AccountPage({Key? key}) : super(key: key);
16
17   @override
18   State<AccountPage> createState() => _AccountPageState();
19 }
20
21 // ส่วนที่ 5: _MainPageState คือ state ของ MainPage
22 > class _AccountPageState extends State<AccountPage> { ...
912
```

1. สร้างไฟล์ login และ register

- ไฟล์ login เก็บไว้ที่ /lib/src/login.dart
- ไฟล์ register เก็บไว้ที่ /lib/src/register.dart

2. สร้างไฟล์ api_config เพื่อเก็บ API Endpoint

ไฟล์ api_config เก็บไว้ที่ /lib/src/api_config.dart

```
1  class ApiConfig {  
2  |    static const String apiEndpoint = 'http://localhost:4000/api';  
3  }
```

3. แก้ไขไฟล์ /lib/src/account.dart แต่ละส่วน ดังนี้

ดาวน์โหลดได้ที่ : https://drive.google.com/drive/folders/1Psw8cZxwl8zhSuY6UBNnBC57_c71pxui?usp=sharing

- ส่วนที่ 1

```
1 // ส่วนที่ 1: การ import libraries และไฟล์ที่เกี่ยวข้อง
2 import 'package:flutter/material.dart';
3 import 'package:http/http.dart' as http;
4 import 'package:jwt_decoder/jwt_decoder.dart';
5 import 'dart:convert';
6 import 'package:shared_preferences/shared_preferences.dart';
7 import 'package:intl/intl.dart';
8 import 'package:image_picker/image_picker.dart';
9 import 'login.dart';
10 import 'register.dart';
11 import 'api_config.dart';
```

- ส่วนที่ 4 (คงเดิม)

```
12
13 // ส่วนที่ 4: MainPage คือ StatefulWidget ซึ่งสามารถเปลี่ยนแปลงค่าหรือสถานะได้
14 class AccountPage extends StatefulWidget {
15   const AccountPage({Key? key}) : super(key: key);
16
17   @override
18   State<AccountPage> createState() => _AccountPageState();
19 }
20
```

• ส่วนที่ 5

```
21 // ส่วนที่ 5: _MainPageState คือ state ของ MainPage
22 class _AccountPageState extends State<AccountPage> {
23     // Define primary color
24     final Color primaryColor = const Color(0xFF2F80ED);
25
26     // Define user information
27     String _username = '';
28     String _firstName = '';
29     String _lastName = '';
30     String _email = '';
31     String _address = '';
32     String _mobilePhone = '';
33     String _gender = '';
34     String _birthdate = '';
35     String? _imageUrl;
36     bool _isLoading = true;
37     bool _isUploading = false;
38     bool _isEditing = false;
39     bool isUserLoggedIn = false;
40
41     // Define text editing controllers
42     final TextEditingController _usernameController = TextEditingController();
43     final TextEditingController _firstNameController = TextEditingController();
44     final TextEditingController _lastNameController = TextEditingController();
45     final TextEditingController _emailController = TextEditingController();
46     final TextEditingController _addressController = TextEditingController();
47     final TextEditingController _mobilePhoneController = TextEditingController();
48     final TextEditingController _genderController = TextEditingController();
49     final TextEditingController _birthdateController = TextEditingController();
50     final ImagePicker _picker = ImagePicker();
```

```

51
52 // Initialize the state
53 @override
54 void initState() {
55     super.initState();
56     _checkLoginStatus();
57     _fetchUserProfile();
58 }
59
60 // Check and update login status
61 > Future<void> _checkLoginStatus() async { ...
82
83 // Fetch user profile data
84 > Future<void> _fetchUserProfile() async { ...
150
151 // Update user profile
152 > Future<void> _updateUserProfile() async { ...
207
208 // Pick and upload image
209 > Future<void> _pickAndUploadImage() async { ...
328
329 // Map gender integer to string
330 > String _mapGender(int gender) { ...
340
341 // Map gender string to integer
342 > int _getGenderCode(String gender) { ...
352

```

```

353 // Define formatted date for birthdate
354 > String _getFormattedBirthdate() { ...
364
365 // A user clicks the logout button
366 > Future<void> _logout() async { ...
377
378 // A user clicks the register button
379 > Future<void> _register() async { ...
387
388 // Build profile information item
389 > Widget _buildProfileInfoItem(String label, String value) { ...
426
427 // Build profile input field
428 > Widget _buildProfileInputField(TextEditingController controller, String label, IconData icon, {bool isDatePicker = false, bool isGender = false}) { ...
514
515 // Show a snackbar message
516 > void _showSnackBar(String message) { ...
526
527 // Render UI
528 @override
529 > Widget build(BuildContext context) { ...
910
911 }
912

```

ตาราง 4.1 เปรียบเทียบ Flutter Widget กับ HTML tag

Flutter Widget	HTML Tag	คำอธิบาย
Text('Hello')	Hello	แสดงข้อความธรรมดา
TextField()	<input type="text" />	ช่องกรอกข้อความ
ElevatedButton()	<button>	ปุ่มคลิก
Image.asset() / Image.network()		แสดงภาพจาก asset หรือ URL
Column()	<div style="display: flex; flex-direction: column;">	จัด layout แนวตั้ง
Row()	<div style="display: flex; flex-direction: row;">	จัด layout แนวนอน
Container()	<div>	กล่องทั่วไปที่ใช้จัด layout และตกแต่ง
Padding()	<div style="padding: ...">	ใช้เพิ่มระยะห่างภายในองค์ประกอบ
SizedBox(height: x)	<div style="height: xpx">	กล่องเว้นช่องว่างหรือขนาดคงที่
Scaffold()	<body>	โครงสร้างพื้นฐานของหน้า UI
AppBar()	<header>	ส่วนหัวของหน้า
BottomNavigationBar()	<footer> + <nav>	แถบเมนูด้านล่าง
ListView()	หรือ <div style="overflow: scroll;">	รายการเลื่อนขึ้นลงได้
Checkbox()	<input type="checkbox">	ช่องติ๊กเลือก
Radio()	<input type="radio">	ปุ่มตัวเลือกเดียว
Switch()	<input type="checkbox" role="switch">	ปุ่มเปิด/ปิด

Flutter Widget	HTML Tag	คำอธิบาย
Form()	<form>	แบบฟอร์มเก็บข้อมูล
DropDownButton()	<select><option></option></select>	รายการแบบ dropdown
Navigator.push()	 หรือ JS Routing	การเปลี่ยนหน้าในแอป
AlertDialog()	window.alert() หรือ <dialog>	กล่องข้อความแจ้งเตือน
Column()	<div style="display: flex; flex-direction: column;">	จัด layout แนวตั้ง

ตาราง 4.2 เปรียบเทียบ Flutter Widget (Property) กับ TailwindCSS (class name)

Flutter (Property / Method)	Tailwind CSS Class	คำอธิบาย
Container(height: 120, color: primaryColor)	h-[120px] bg-[#2F80ED]	ความสูง 120px, สีพื้นหลังตามสี primary
margin: EdgeInsets.only(top: 10)	mt-2.5	Margin top 10px (2.5*4px)
Center(child: ...)	flex justify-center items-center	จัดกึ่งกลางทั้งแนวแกน x และ y
Column(children: [...])	flex flex-col	เรียงลูกแนวตั้ง
Row(...)	flex flex-row	เรียงลูกแนวนอน
Text("...")	text-base หรือ text-lg ฯลฯ	ขนาดตัวอักษร
fontWeight: FontWeight.bold	font-bold	ตัวหนา
color: Colors.grey.shade600	text-gray-600	สีเทาระดับ 600
EdgeInsets.symmetric(horizontal: 16)	px-4	padding แนวนอน (16px = 4 * 4px)
BoxDecoration(borderRadius: BorderRadius.circular(16))	rounded-lg	มุมโค้ง 16px
BoxDecoration(boxShadow: [...])	shadow-md / shadow	เงา
ElevatedButton	btn หรือ bg-blue-500 text-white px-4 py-2 rounded	ปุ่มพื้นหลังสีฟ้า ขอบโค้ง
SingleChildScrollView	overflow-y-scroll หรือ overflow-auto	ให้ scroll ได้
CircleAvatar(radius: 50)	w-[100px] h-[100px] rounded-full	รูปวงกลมขนาด 100px
Padding(padding: EdgeInsets.all(16))	p-4	padding ทั้งหมด 16px
const SizedBox(height: 16)	mt-4 หรือ mb-4	ระยะห่างบน/ล่าง

Flutter (Property / Method)	Tailwind CSS Class	คำอธิบาย
SafeArea	pt-safe (ไม่มีตรงตัว ต้องจัด padding บนเอง)	เว้นพื้นที่ตามขอบปลอดภัย
TextField / Input	input input-bordered หรือ border p-2 rounded	กล่องกรอกข้อมูล
Stack(children: [...])	relative (ใช้ absolute กับลูก)	ซ้อน widget
Positioned(bottom: 0, right: 0)	absolute bottom-0 right-0	วางตำแหน่ง widget ซ้อน

มหาวิทยาลัยเทคโนโลยีราชมงคลตะวันออก
คณะบริหารธุรกิจและเทคโนโลยีสารสนเทศ
สาขาวิชาวิทยาการคอมพิวเตอร์



Thankyou
For Your Attention

busit.rmutto.ac.th

